

周振轩：压轴选择题与数据结构大题综合卷

使用建议：这份不按一次课完成设计。每次课后剩余 15-25 分钟，可以做 2-3 道压轴选择题，或拆一题数据结构大题做“读题-画指针/队列/栈-填空-复盘”。

训练重点：压轴选择题重点追踪二分边界、链表指针、栈队列状态；大题重点追踪

head/p/q/next/-1、队首队尾、栈顶变化，判断插入、删除、合并、逆置、出入队/栈时哪个变量先改。

A组：压轴选择题

A1：链表判环与入口定位

出处：[202510-强基联盟-高三-月考-11](#)。（来源：[202510-强基联盟-高三-月考.pdf](#)；难度：中等）

11. 以下 Python 程序用“双指针”检测链表中的环并找到环的起始点，快指针（fast）步进为 2，慢指针（slow）步进为 1，若存在环，快慢指针一定会在环内相遇。

```
gra = [['水星', 1], ['金星', 2], ['地球', 3], ['火星', 4], ['木星', 2]]
head = 0
slow = fast = head
while True:
    slow = gra[slow][1]
    fast = ____ (1)
    if slow == fast:
        meetn = slow
        break
p = ____ (2)
q = meetn
while p != q:
    p = gra[p][1]
    q = gra[q][1]
print(gra[p][0])
```

运行程序，输出结果为“地球”，上述程序中划线 (1)(2) 处可选的代码有：

- ① `gra[fast][1]`
- ② `gra[gra[fast][1]][1]`
- ③ `head`
- ④ `meetn`

则 (1)(2) 处正确的语句顺序是 ()

- A. ②③
- B. ②④
- C. ①③
- D. ①④

答案

正确答案： A

解析

链表结构：水星(0)→金星(1)→地球(2)→火星(3)→木星(4)→地球(2)，环入口为地球(索引 2)。

(1) 快指针每次走两步：第一步 `gra[fast][1]` 到下一节点，第二步 `gra[gra[fast][1]][1]` 再到下下节点，对应代码②。

(2) Floyd 判环算法：快慢指针相遇后，将一个指针重置到链表头 `head`，两指针同步走，再次相遇点即为环入口，对应代码③。

故 (1) 选②，(2) 选③，答案为 ②③ → A。

A2：链表局部逆置

出处：[202605-浙江七校-高三-月考-11](#)。（来源：[202605-浙江七校-高三-月考.pdf](#)；难度：中等）

使用列表 `lst` 模拟链表结构，每个节点由数据区域和指针区域组成，变量 `h` 为头指针。函数 `move(p,m)` 用于实现节点的局部逆序：将节点 `p` 之后的连续 `m+1` 个节点进行逆序链接。若调用函数 `move(3,2)` 后，链表状态由第 11 题图 a 调整为第 11 题图 b。实现该功能的 Python 程序如下：

	数据区域	指针区域		数据区域	指针区域
0	20	5	h→0	20	2
→1	26	3	1	26	3
2	4	0	2	4	4
3	23	2	3	23	5
4	2	-1	4	2	-1
5	36	4	5	36	0

第 11 题图 a

第 11 题图 b

```
def move(p,m):
    c = lst[p][1]
    for i in range(m):
```

```

t = lst[c][1]
lst[c][1] = lst[t][1]
-----①-----
-----②-----

```

(1) (2) 划线处可选填的有：① `lst[p][1] = t`；② `lst[t][1] = lst[p][1]`；③ `lst[t][1] = c`；④ `lst[p][1] = lst[t][1]`。则划线处应依次填入的是 ()

- A. ①②
- B. ①③
- C. ②①
- D. ②④

答案

正确答案： C

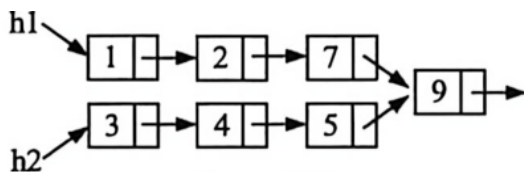
解析

局部逆置时先让待移节点 `t` 指向当前 `p` 后继，再让 `p` 指向 `t`，即先填 `lst[t][1]=lst[p][1]`，再填 `lst[p][1]=t`。

A3：两条有序链表合并

出处： [202604-精诚联盟-高三-二模-12](#)。（来源： [202604-精诚联盟-高三-二模.pdf](#)；难度：中等）

12. 使用列表 `d` 来模拟链表结构，存在若干节点，每个节点由数据域和指针域组成，如第 12 题图 a 所示，由头指针 `h1` 和 `h2` 起始的两条链表各节点均按数据域升序，其逻辑顺序的最后一个节点相同。已知节点 `h1` 的数据域小于节点 `h2` 的数据域，现要将两条链表合并为一条升序链表，如第 12 题图 b 所示。实现该功能的程序段如下，则加框处应填入的正确代码为 ()



第 12 题图 a



第 12 题图 b

```

h = h1
p = 0
h2 = q = 6
while p != q:
    t = d[p][1]
    while d[t][0] < d[q][0]:
        p = t

```

```
t = d[t][1]
# 加框处
```

- A.

```
d[p][1] = q
p = q
q = t
```

- B.

```
d[p][1] = q
p = t
q = d[q][1]
```

- C.

```
d[q][1] = t
p = q
q = t
```

- D.

```
d[q][1] = t
p = t
q = d[q][1]
```

答案

正确答案： A

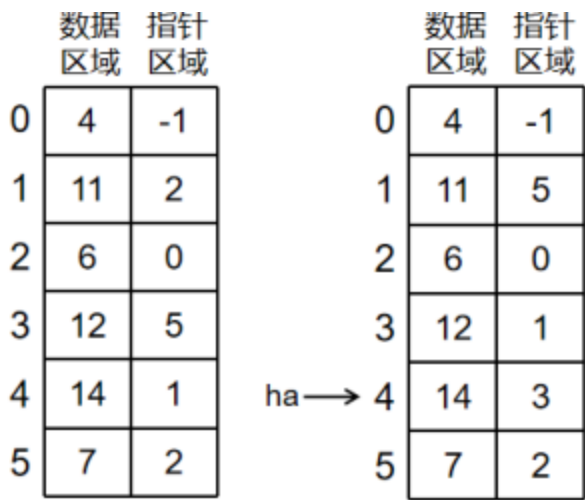
解析

合并时 p 停在第一条链表中最后一个比 q 小的节点，需先令 $d[p][1]=q$ 把 q 接入 p 后，再把 p 更新到 q，q 更新到原来 p 的后继 t，继续比较和插入。

A4：相交链表融合

出处：[202605-北斗星盟-高二-月考-12](#)。（来源：[202605-北斗星盟-高二-月考.pdf](#)；难度：中等）

使用列表 d 模拟链表结构，每个节点包含数据区域和指针区域。如第 12 题图 a 所示， ha 和 hb 分别为两个链表的头指针，两个链表的节点均降序排列，无重复节点，且两个链表在某个节点相交，现要将两个链表按照节点数据区域降序融合为一个链表，结果如第 12 题图 b 所示，实现该功能的程序段如下，划线处应填入的正确代码为（ ）



第 12 题图 a

第 12 题图 b

```
q=ha; k=hb
p=d[q][1]
flag=False
while not flag:
    num=d[k][0]
    t=d[k][1]
    if k!=ha and num>d[ha][0]:
        d[k][1]=ha; ha=k
    elif k==ha:
        flag=True
    else:
        while p!=-1 and d[p][0]>num and ____①____:
            q=p; p=d[p][1]
        if p==k:
            flag=True
        else:
            d[q][1]=k
            d[k][1]=p
        ____②____
```

- A. ① p!=k ② k=d[k][1]
- B. ① p!=k ② k=t
- C. ① q!=k ② k=d[k][1]
- D. ① q!=k ② k=t

答案

正确答案： D

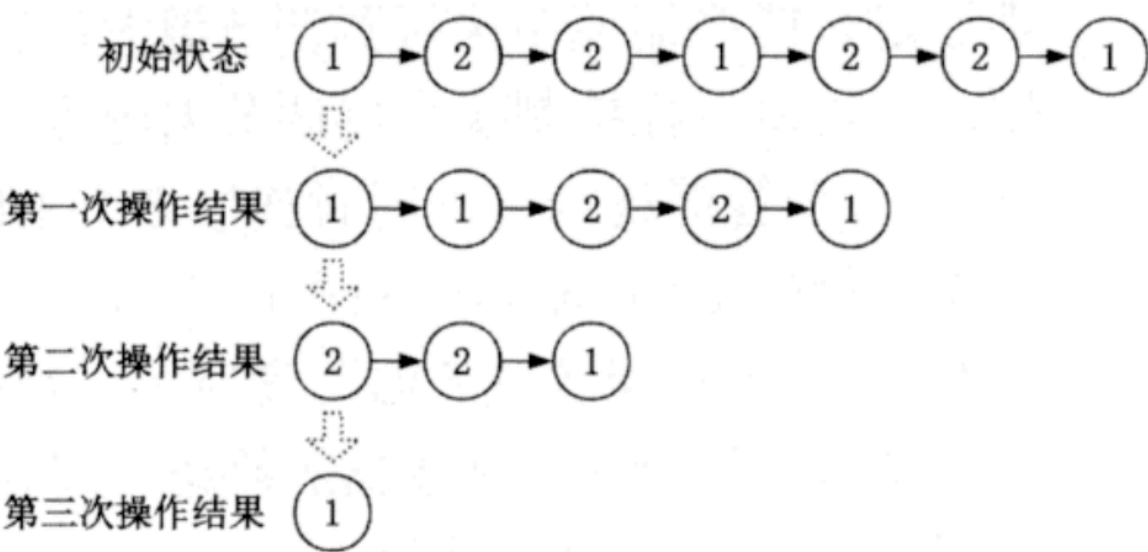
解析

遍历插入时应防止 q 到达相交节点 k，避免破坏共享后续链表，因此条件为 $q \neq k$ 。循环末尾要按原链表保存的后继 t 继续处理，故 $k=t$ 。

A5：连续相同节点删除

出处：[202506-杭州-高二-期末-12](#)。（来源：[202506-杭州-高二-期末.pdf](#)；难度：困难）

12. 使用列表 d 模拟链表结构，每个节点包含数据区域和指针区域，h 为头指针。现要从链表中找出节点值相同的连续节点并删除，重复执行以上操作，直到链表中不存在节点值相同的连续节点。删除过程如第 12 题图所示。实现该功能的 Python 程序段如下：



第 12 题图

```
p = h
while d[p][1] != -1 and d[h][1] != -1:
    pre = p = h
    q = d[p][1]
    while q != -1 and d[q][0] != d[p][0]:
        pre = p; p = q; q = d[q][1]
    while q != -1 and d[q][0] == d[p][0]:
        ----
```

```
if p == pre:
    h = q
```

程序中加框处应填入的语句分别为 ()

- A. `d[pre][1] = d[p][1]; q = d[p][1]`
- B. `d[pre][1] = d[q][1]; q = d[q][1]`
- C. `d[p][1] = d[q][1]; q = d[p][1]`
- D. `d[p][1] = d[q][1]; q = d[q][1]`

答案

正确答案： B

解析

p 指向当前连续相同节点段的第一个节点，q 指向其后相同值节点。若这段节点不在表头，应让前驱 pre 的指针跳过当前相同节点段，因此要不断执行 `d[pre][1] = d[q][1]`，再令 `q = d[q][1]` 继续向后检查同值节点。若连续段从头节点开始，循环后由 `if p == pre: h = q` 更新头指针。

A6：链表连续递增段

出处：[202605-强基联盟-高三-月考-12](#)。（来源：[202605-强基联盟-高三-月考.pdf](#)；难度：中等）

12. 使用列表 a 模拟链表结构，每个节点由数据域和指针域组成，head 为头指针。现需要找出链表中最长连续递增子序列的所有节点数据域之和，实现该功能的 Python 程序如下：

```
a = [[3, 1], [5, 2], [2, 3], [4, 4], [6, 5], [1, 6], [3, 7], [4, 8], [5, 9],
[7, 10], [2, -1]]
p = head = 0
max_n = n = 1
max_s = s = a[head][0]
while a[p][1] != -1:
    q = a[p][1]
    if a[p][0] < a[q][0]:
        n += 1
        -----①-----
        if n > max_n:
            max_n = n
            max_s = s
    else:
        n = 1
```

-----②-----
-----③-----

划线处有如下可选代码：

- ① `p = q -`
- ② `p = a[p][1] -`
- ③ `s = a[q][0] -`
- ④ `s += a[q][0]` 则划线处应填入的正确代码为
- A. ①②④
- B. ④③①
- C. ①③④
- D. ②④①

答案

正确答案： B

解析

递增时当前段长度加 1，段和应累加新节点数据 `s += a[q][0]`；递增中断时新段从 q 开始，`s = a[q][0]`；每轮结束都将指针推进到 q。

A7：链表随机调整与不可能输出

出处：[202605-诸暨-高三-三模-12](#)。（来源：[202605-诸暨-高三-三模.pdf](#)；难度：中等）

12. 有如下 Python 程序段：

```
import random

a = [[1, 5], [5, 2], [2, 4], [2, 0], [4, -1], [3, 1]]
p = head = 3
q = a[p][1]
x = random.randint(1, 5)
while q != -1 and a[q][0] < x:
    p, q = q, a[q][1]
t = p
while q != -1:
    if a[q][0] < x:
        a[p][1] = a[q][1]
        a[q][1] = a[t][1]
        a[t][1] = q
        t, q = a[t][1], a[p][1]
```



```
else:
    p = q
    q = a[q][1]
# 遍历链表 a，依次输出 a[i][0]，代码略
```

执行上述程序后，下列输出结果不可能的是

- A. 2, 1, 3, 5, 2, 4
- B. 2, 1, 2, 3, 5, 4
- C. 2, 1, 3, 2, 5, 4
- D. 2, 1, 2, 3, 4, 5

答案

正确答案： D

解析

程序围绕随机值 x 调整链表中小于 x 的节点位置，但不会破坏原链表中部分节点可达关系到产生 D 所示全局升序序列，因此 D 不可能。

A8：链表遍历逻辑错误

出处：[202604-稽阳联谊-高三-期中-12](#)。（来源：[202604-稽阳联谊-高三-期中.pdf](#)；难度：中等）

用列表模拟链表，每个元素包含两个字段，依次表示数据和指针，分别统计从列表每个元素开始的链表中偶数的数量。如 $a = [[37, 1], [26, 5], [35, 3], [6, 4], [18, -1], [29, -1]]$ 中，由 $a[0]$ 开始的链表 $[37, 1] \rightarrow [26, 5] \rightarrow [29, -1]$ 中有一个偶数 26。实现上述功能的 Python 代码如下，标号 ①②③④ 语句中有逻辑问题的是（ ）

```
rst = [0]*11; p = i = 0
while i < len(a):                                #①
    if p != -1:
        if a[p][0] % 2 == 0:
            rst[i] += 1                            #②
            p = a[p][1]                            #③
    else:
        print("该位置开始的链表中偶数个数有：", rst[i])
        i += 1
        p = i                                    #④
```

- A. ①
- B. ②

- C. ③
- D. ④

答案

正确答案： C

解析

`p = a[p][1]` 用来沿指针移动，应在判断当前节点是否为偶数之后无条件执行。题中它缩进在偶数判断内部，遇到奇数节点时 `p` 不变，会导致链表遍历停滞。

A9：旋转有序数组二分查找

出处：[202510-强基联盟-高三-月考-12](#)。（来源：[202510-强基联盟-高三-月考.pdf](#)；难度：困难）

12. 对非降序数组 `nums` 循环右移一定位数，构建新的数组，如 `nums=[1, 2, 3, 4, 5]`，循环右移 3 位后新数组为 `[3,4,5,1,2]`。以下 Python 程序尝试在循环右移后的数组中查找到目标值 `key`。

```
def search(nums, key):
    i, j = 0, len(nums)-1
    while i <= j:
        m = (i+j)//2
        if nums[m] == key:
            return True
        if nums[i] <= nums[m]:
            if nums[i] <= key < nums[m]:
                j = m-1
            else:
                i = m+1
        else:
            if nums[m] < key <= nums[j]:
                i = m+1
            else:
                j = m-1
    return False
```

调用函数，以下哪组数据的返回值与其它三项不同（ ）

- A. `nums = [3, 1], key = 1`
- B. `nums = [2, 3, 4, 5, 6], key = 2`
- C. `nums = [1, 0, 1, 1, 1], key = 0`

- D. `nums = [4, 5, 6, 7, 0, 1, 2]`, `key = 4`

答案

正确答案： C

解析

逐一验证：

A. `nums=[3,1]`, `key=1` : `i=0,j=1,m=0`。 `nums[0]=3≠1`, `nums[0]≤nums[0]` (`3≤3`) , `key=1` 不满足 `nums[0]≤key<nums[0]` (`3≤1<3`) , `i=1`。 `m=1`, `nums[1]=1==key` → True。

B. `nums=[2,3,4,5,6]`, `key=2` : 未旋转。 `i=0,j=4,m=2`, `nums[2]=4>2`, `4≤4?` Yes, `2≤2<4?` Yes, `j=1`。 `m=0`, `nums[0]=2==key` → True。

C. `nums=[1,0,1,1,1]`, `key=0` : `i=0,j=4,m=2`。 `nums[2]=1≠0`, `nums[0]≤nums[2]` (`1≤1`) , `key=0` 不满足 `1≤0<1`, `i=3`。 `m=3`, `nums[3]=1≠0`, `nums[3]≤nums[3]` (`1≤1`) , `0` 不满足 `1≤0<1`, `i=4`。 `m=4`, `nums[4]=1≠0`, `nums[4]≤nums[4]` (`1≤1`) , `0` 不满足 `1≤0<1`, `i=5 > j`, 返回 False。

(注：该算法在存在重复元素时可能失效。)

D. `nums=[4,5,6,7,0,1,2]`, `key=4` : 标准搜索旋转数组。 `i=0,j=6,m=3`, `nums[3]=7≠4`, `nums[0]≤nums[3]` (`4≤7`) , `4≤4<7?` Yes, `j=2`。 `m=1`, `nums[1]=5≠4`, `4≤5?` Yes, `4≤4<5?` Yes, `j=0`。 `m=0`, `nums[0]=4==key` → True。

只有 C 返回 False, 其余均返回 True。

A10：二分查找数对与步数统计

出处： [202511-杭州-高三-一模-12](#)。（来源： [202511-杭州-高三-一模.pdf](#)；难度：困难）

12. 有如下 Python 程序段：

```
ans = 0
for i in range(len(a)):
    L, H = i+1, len(a)-1
    p = 0
    while L <= H:
        m = (L+H) // 2
        p += 1
        if a[m] == t-a[i]:
            ans += p
            break
        if a[m] > t-a[i]:
```

```
H = m-1
else:
    L = m+1
```

若 a 为 $[3, 4, 8, 15, 24, 27, 32, 33, 47]$ ， t 为 35，运行该程序段后，变量 ans 的值为（）

- A. 3
- B. 5
- C. 6
- D. 11

答案

正确答案： A

解析

程序在已排序数组 a 中查找所有满足 $a[i] + a[j] = t$ ($j > i$) 的数对， ans 累计每次查找所需的二分查找步数 p 。

$t=35$, $a = [3,4,8,15,24,27,32,33,47]$

对每个 i ，在 $i+1$ 到 $\text{len}(a)-1$ 中二分查找 $\text{target} = t-a[i]$ ：

- $i=0$, $a[0]=3$, $\text{target}=32$ ： $L=1, H=8$ 。 $m=4(a[4]=24 < 32)$, $L=5$ 。 $m=6(a[6]=32)=32$ ，找到。 $p=2$ 。 $\text{ans}=2$ 。
- $i=1$, $a[1]=4$, $\text{target}=31$ ： 查找...31 不在数组中，无累加。
- $i=2$, $a[2]=8$, $\text{target}=27$ ： $L=3, H=8$ 。 $m=5(a[5]=27)=27$ ，找到。 $p=1$ 。 $\text{ans}=3$ 。
- $i=3$, $a[3]=15$, $\text{target}=20$ ： 不在，无贡献。
- $i=4\sim 8$ 的 $a[i] \geq 24$ ， $\text{target} \leq 11$ ，位于数组前部但 $L > H$ 区段，均无后续找到。

最终 $\text{ans}=3$ 。

A11：二分查找递归与循环等价

出处：[202512-嘉兴-高三-月考-12](#)。（来源：[202512-嘉兴-高三-月考-试卷.pdf](#)；难度：困难）

12. 在非降序有序数组 a 中查找元素 key 所处的位置， L 、 R 为查找时数组 a 的左右边界位置。若函数 f_1 、 f_2 功能相同，则函数 f_2 加框处的正确代码为（）

```
def f1(a, key, L, R):
    if L > R:
        return -1
```

```

M = (L + R) // 2
if a[M] == key:
    if M == L or a[M - 1] != key:
        return M
    else:
        return f1(a, key, L, M - 1)
elif key > a[M]:
    return f1(a, key, M + 1, R)
else:
    return f1(a, key, L, M - 1)

def f2(a, key, L, R):
    while L < R:
        M = (L + R) // 2
        # 加框处代码
    if a[L] == key:
        return L
    else:
        return -1

```

- A. `if key > a[M]: L = M + 1 else: R = M`
- B. `if key > a[M]: L = M else: R = M - 1`
- C. `if key > a[M]: L = M + 1 else: R = M - 1`
- D. `if key > a[M]: L = M else: R = M + 1`

答案

正确答案： A

解析

f1 的功能：在非降序有序数组中**查找 key 第一次出现的位置**。当 `a[M] == key` 时，若 M 已经是左边界或前一个元素不等于 key，则返回 M（即为首次出现）；否则继续在左半部分查找。

f2 需要实现相同功能。f2 的循环条件是 `while L < R`，出循环时 `L == R`，然后检查 `a[L] == key`。

对于非降序有序数组中查找首次出现位置的标准二分模板，当 `key > a[M]` 时目标在右半部分，`L = M + 1`；否则（`key <= a[M]`）目标在左半部分（包含 M），`R = M`。

A 正确： `if key > a[M]: L = M + 1 else: R = M` → 这是查找首次出现位置的标准写法。

B 错误： `L = M` 可能导致死循环（当 `L = M` 时）。

C 错误： `R = M - 1` 可能跳过答案（当 `a[M] == key` 时可能丢失首次出现位置）。

D 错误： $R = M + 1$ 完全错误的分支逻辑。

A12：山脉数组二分查找

出处：202512-Z20名校联盟-高三-月考-12。（来源：202512-Z20名校联盟-高三-月考-试卷.pdf；难度：中等）

12. 已知一个长度为 n ($n \geq 3$) 的整数数组 a 满足以下条件：数组元素大小呈先升后降，仅存在一个索引 i ($0 < i < n-1$) 使得 $a[i-1] < a[i]$ ，且 $a[i] > a[i+1]$ 。这样的数组被称为山脉数组。定义以下函数实现在山脉数组中找到峰值元素（即最大值）的索引。

```
def search(a):
    left, right = 0, len(a) - 1
    while left < right:
        mid = (left + right) // 2
        if ①:
            ②
        else:
            ③
    return left
```

划线①②③处应填入的代码为（ ）

- A. ① $a[mid] > a[mid + 1]$ ② $right = mid + 1$ ③ $left = mid$
- B. ① $a[mid] < a[mid + 1]$ ② $left = mid + 1$ ③ $right = mid$
- C. ① $a[mid] < a[mid + 1]$ ② $right = mid + 1$ ③ $left = mid$
- D. ① $a[mid] > a[mid + 1]$ ② $left = mid + 1$ ③ $right = mid$

答案

正确答案： B

解析

山脉数组先升后降，峰值在上升段的最右端（或下降段的最左端）。

二分查找峰值策略：比较 $a[mid]$ 和 $a[mid+1]$ ：

- 若 $a[mid] < a[mid+1]$ ：说明 mid 在上升段，峰值在右侧 $\rightarrow left = mid + 1$
- 若 $a[mid] > a[mid+1]$ ：说明 mid 在下降段（或就是峰值），峰值在左侧（含 mid ） $\rightarrow right = mid$

① $a[mid] < a[mid + 1]$ ：判断是否在上升段。

② $left = mid + 1$ ：峰值在右侧。

③ `right = mid`：峰值在左侧（含 `mid`）。

选项 A/D 用 `a[mid] > a[mid+1]` 反转了逻辑会导致错误的收缩方向。

选项 C 的②③赋值错误。

A13：旋转序列中的二分查找

出处：[202604-绍兴-高三-期中-12](#)。（来源：[202604-绍兴-高三-期中.pdf](#)；难度：中等）

12. 某队列初始状态为升序序列（序列中的元素互不相同），经过若干次“将队首元素移至队尾”的操作后，所有元素依次出队得到序列 `a`。编写 Python 程序，在序列 `a` 中查找目标值 `key` 的位置，部分代码如下：

```
i, j = 0, len(a) - 1
ans = -1
while i <= j:
    m = (i + j) // 2
    if a[m] == key:
        ans = m
        break
    if _____:
        if a[i] <= key < a[m]:
            j = m - 1
        else:
            i = m + 1
    else:
        if a[m] < key <= a[j]:
            i = m + 1
        else:
            j = m - 1
```

则程序中划线处的代码应为（ ）

- A. `a[i] <= a[m]`
- B. `a[m] <= a[j]`
- C. `a[i] < a[m]`
- D. `a[i] <= a[j]`

答案

正确答案：A

解析

`a[i] <= a[m]` 用来判断左半段是否有序。若左半段有序，再根据 `key` 是否落在 `[a[i], a[m]]` 中调整边界；否则右半段有序，按右半段范围调整。

A14：满二叉查找路径统计

出处：[202604-台州-高三-二模-11](#)。（来源：[202604-台州-高三-二模.pdf](#)；难度：中等）

11. 定义如下函数：

```
def search(a, key):
    i = 0
    j = len(a) - 1
    while i <= j:
        m = (i + j) // 2
        if a[m] == key:
            break
        elif a[m] > key:
            j = m - 1
        else:
            i = m + 1
    return i == j
```

列表 `a` 中有 1023 个互不相同的整数且按升序排列，若调用该函数返回值为 `True`，则列表 `a` 中符合 `key` 的元素个数为（ ）

- A. 128
- B. 256
- C. 512
- D. 1023

答案

正确答案：C

解析

函数只有在找到 `key` 时跳出循环，并返回当时的 `i==j`。长度 1023 的有序表构成满二分查找区间，只有最后缩小到单元素区间时找到的元素会使 `i==j` 为 `True`，这类叶子位置共有 512 个。

A15：二分查找访问路径判断

出处：[202605-浙江七校-高三-月考-12](#)。（来源：[202605-浙江七校-高三-月考.pdf](#)；难度：中等）

有如下 Python 程序段：


```

a=[20,18,17,17,12,12,12,11]
key=int(input("key="))
q=[0]*(len(a)+1)
L=R=len(a)//2
i,j=0,len(a)-1
while i<=j:
    m=(i+j)//2
    if a[m]>key:
        i=m+1
        R+=1; q[R]=m
    else:
        j=m-1
        L-=1; q[L]=m
print(q[L:R+1])

```

程序运行时，输入 key 值，则输出结果不可能为（ ）

- A. [1,3,0,0]
- B. [6,0,3,5]
- C. [4,5,0,3]
- D. [7,0,3,5,6]

答案

正确答案： B

解析

程序记录二分查找访问过的中点下标，并按比较结果放到左右两侧。对所有可能 key 枚举访问路径，B 中的访问顺序与二分区间变化不一致，因此不可能输出。

A16：二分查找边界变量判断

出处： [202606-九校-高二-期末-12](#)。（来源： [202606-九校-高二-期末.pdf](#)；难度：中等）

12. 有如下 Python 程序段：

```

a = [11, 15, 15, 17, 19, 19, 19, 21, 26, 28]
i = 0
j = 9
c = 0
key = int(input())
while i <= j:
    m = (i + j) // 2

```

```

c += 1
if a[m] > key:
    j = m - 1
else:
    i = m + 1
print(j)

```

执行该程序段后，下列说法不正确的是（ ）

- A. 若输入 20，程序运行后输出的值 j 是 6
- B. 程序运行后，a[i] 的值可能等于 key
- C. 程序的功能是输出列表中最后一个小于等于 key 的元素所在的位置
- D. 对于不同的输入值 key，程序运行后 c 的值一定大于 2

答案

正确答案： B

解析

循环结束时，j 指向最后一个小于等于 key 的元素位置，i 指向第一个大于 key 的位置，因此 a[i] 一般不会等于 key。输入 20 时最后一个小于等于 20 的元素为下标 6；该程序是二分查找变形；该长度列表的有效查找过程比较次数均大于 2。

A17：随机栈过程追踪

出处：[202603-新阵地-高三-月考-12](#)。（来源：[202603-新阵地-高三-月考.pdf](#)；难度：困难）

12. 有如下 Python 程序段：

```

from random import randint
nums = [3, 1, 4, 2]
stack = []
while len(nums) > 0:
    if randint(0, 1) == 0: # randint(a,b)随机生成一个[a,b]范围内的整数
        nums.append(nums.pop(0))
    else:
        if len(stack) > 0 and nums[0] > stack[-1]:
            stack[-1] = nums.pop(0)
        else:
            stack.append(nums.pop(0))

```

运行该程序，输出的 stack 列表不可能为（ ）

- A. [4, 2]

- B. [2,4]
- C. [4,1]
- D. [1,4]

答案

正确答案： D

解析

当取出的新元素大于栈顶时会替换栈顶，否则追加到栈尾；同时 `nums` 可轮转。通过跟踪可能的取出顺序可知 A、B、C 都可构造，[1,4] 不可能成为最终栈。

A18：栈过程与输出判断

出处：202512-精诚联盟-高三-月考-12。（来源：202512-精诚联盟-高三-月考.pdf；难度：困难）

“气温跨度”定义为这一天之前连续多少天的最高气温低于或等于这一天的最高气温。

	周一	周二	周三	周四	周五	周六	周日
最高气温	10°C	17°C	13°C	11°C	11°C	15°C	16°C
气温跨度	1	2	1	1	2	4	5

实现该功能的Python 程序段如下，方框中应填入的正确代码为

```
d = [10,17,13,11,11,15,16]          #存放每天的最高气温
k = [1 for i in range(len(d))]      #存放每天的气温跨度
s = [0 for i in range(len(d))]
top = -1

for i in range(1, len(d)):
    while top != -1 and d[s[top]] <= d[i]:
        top -= 1
    ▲
print(k)
```

- A.

```
top += 1
s[top] = i
if top == -1:
```

```
    k[i] = i + 1
else:
    k[i] = i - s[top]
```

- B.

```
if top == -1:
    k[i] = i + s[top]
else:
    k[i] = i - 1
top += 1
s[top] = i
```

- C.

```
if top == -1:
    k[i] = i - s[top]
else:
    k[i] = i + 1
top += 1
s[top] = i
```

- D.

```
if top == -1:
    k[i] = i + 1
else:
    k[i] = i - s[top]
top += 1
s[top] = i
```

答案

正确答案： D

解析

栈 s 中保存仍可能作为跨度边界的日期索引。弹出所有不高于当天气温的索引后，若 $top == -1$ ，说明从第 0 天到第 i 天都满足条件，跨度为 $i+1$ ；否则跨度为 $i-s[top]$ 。计算完后再把当天索引入栈，所以 D 正确。

A19：队列指针与过程判断

出处: [202511-湖丽衢-高三-一模-12](#)。(来源: [202511-湖丽衢-高三-一模.pdf](#); 难度: 中等)

12. 有如下 Python 程序段:

```
def max_w(nums, k):
    dq = []
    result = []
    for i in range(len(nums)):
        while len(dq) > 0 and nums[dq[-1]] < nums[i]:
            dq.pop() # 去除 dq 最后一个元素
        dq.append(i) # 为 dq 添加一个元素 i
        if len(dq) > 0 and i - dq[0] >= k:
            dq.pop(0) # 去除 dq 第一个元素
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result

nums = [1, 3, -1, -3, 5, 3, 6, 7]; k = 3
print(max_w(nums, k))
```

运行该程序段后, 输出的结果是 ()

- A. [3, 3, 5, 5, 6, 7]
- B. [-1, -3, -3, -3, 3, 3]
- C. [3, 3, -1, 5, 5, 6]
- D. [1, 3, 3, 3, 5, 5, 6, 7]

答案

正确答案: A

解析

这是滑动窗口最大值的标准解法, 使用单调队列维护窗口内的最大值候选。k=3, 数组长度 8, 窗口数为 6。

逐窗口分析:

- i=0 : dq=[0], nums[0]=1
- i=1 : 3>1, dq=[1], nums[1]=3
- i=2 : -1<3, dq=[1,2], i>=2, result=[3]
- i=3 : -3<-1, dq=[1,2,3], 1-1>=3 ? 0>=3 否, result=[3,3]
- i=4 : 5>3, dq=[4], 4-4>=3 ? 0>=3 否, result=[3,3,5]
- i=5 : 3<5, dq=[4,5], 5-4>=3 ? 1>=3 否, result=[3,3,5,5]

- $i=6$: $6>3>5$, $dq=[6]$, $6-6\geq 3$? $0\geq 3$ 否, $result=[3,3,5,5,6]$
- $i=7$: $7>6$, $dq=[7]$, $7-7\geq 3$? $0\geq 3$ 否, $result=[3,3,5,5,6,7]$

结果为 $[3, 3, 5, 5, 6, 7]$, 选 A。

A20: 贪心距离程序追踪

出处: [202603-宁波十校-高三-月考-12](#)。(来源: [202603-宁波十校-高三-月考.pdf](#); 难度: 中等)

12. 有如下 Python 程序段:

```
# 获取t的值, 代码略
a=[1,2,4,8,9,11,15,18]
for i in range(a[len(a)-1]-a[0],-1,-1):
    tot=1;num=0
    for j in range(1,len(a)):
        if a[j]-a[num]>=i:
            tot+=1
            num=j
    if tot>=t:
        break
```

执行该程序段后, i 的值为 3, 则下列选项中不可能为 t 的值的是 ()

- A. 3
- B. 4
- C. 5
- D. 6

答案

正确答案: A

解析

程序从大到小枚举间距 i , 并贪心统计相邻选中元素差值至少为 i 时最多能选出的元素个数 tot 。若 $t=3$, 例如可选 $1, 8, 15$, 最小间距至少为 7, 因此程序会在大于 3 的间距处提前结束, 不可能得到 $i=3$ 。而 $t=4, 5, 6$ 时在 $i=3$ 可满足, 且更大的 i 不满足。

A21: 约瑟夫环循环淘汰

出处: [202501-湖州-高二-期末-12](#)。(来源: [202501-湖州-高二-期末.pdf](#); 难度: 中等)

12. 有如下 Python 程序段:

```
s = "红橙黄绿青蓝紫"
x = 2
while len(s) > 1:
    x = (x + 3) % len(s)
    s = s[:x] + s[x+1:]
print(s)
```

执行该程序段后，输出的内容是（ ）

- A. 红
- B. 橙
- C. 绿
- D. 蓝

答案

正确答案： C

解析

这是 Josephus 式淘汰，每轮先更新下标，再删除当前位置字符。

轮次	删除下标 x	删除字符	剩余字符串
1	5	蓝	红橙黄绿青紫
2	2	黄	红橙绿青紫
3	0	红	橙绿青紫
4	3	紫	橙绿青
5	0	橙	绿青
6	1	青	绿

最后剩下 绿。

A22：约瑟夫环改编一

改编自 [202501-湖州-高二-期末-12](#)。

有如下 Python 程序段：

```
s = "ABCDE"
x = 0
while len(s) > 1:
```

```
x = (x + 2) % len(s)
s = s[:x] + s[x+1:]
print(s)
```

执行该程序段后，输出的内容是（ ）

- A. A
- B. B
- C. C
- D. D

答案

正确答案： D

解析

轮次	删除下标 x	删除字符	剩余字符串
1	2	C	ABDE
2	0	A	BDE
3	2	E	BD
4	0	B	D

最后剩下 D。

A23：约瑟夫环改编二

改编自 [202501-湖州-高二-期末-12](#)。

有如下 Python 程序段：

```
s = "12345678"
x = 1
while len(s) > 1:
    x = (x + 4) % len(s)
    s = s[:x] + s[x+1:]
print(s)
```

执行该程序段后，输出的内容是（ ）

- A. 2

- B. 4
- C. 6
- D. 8

答案

正确答案： B

解析

轮次	删除下标 x	删除字符	剩余字符串
1	5	6	1234578
2	2	3	124578
3	0	1	24578
4	4	8	2457
5	0	2	457
6	1	5	47
7	1	7	4

最后剩下 4。

B组：链表大题

B1：链表插入、删除与移动操作

出处：[202506-宁波九校-高二-期末-15](#)。（来源：[202506-宁波九校-高二-期末.pdf](#)；难度：困难）

15. 编写一个 Python 程序，实现链表操作：初始链表中的元素从头部到尾部依次为 1 到 20。整数 m 表示后续要进行的操作总数。随后输入的 m 行数据每行包含三个整数，分别记为 a 、 b 和 c 。不同的 a 值对应不同的操作，具体规则如下：

- 当 a 为 1 时，在链表的第 b 个位置插入元素 c ；
- 当 a 为 2 时，将链表中第 b 到第 c 位置的这段子序列进行反转；
- 当 a 为 3 时，删除链表中第 b 到第 c 位置的所有元素；
- 当 a 为 4 时，输出链表中第 b 到第 c 位置的所有元素。

例如，程序运行结果见第 15 题图。

请输入操作总数 m: 9
操作 1: 4 1 8
输出: 1 2 3 4 5 6 7 8
操作 2: 1 3 9
操作 3: 4 2 6
输出: 2 9 3 4 5
操作 4: 2 4 7
操作 5: 4 1 9
输出: 1 2 9 6 5 4 3 7 8
操作 6: 3 1 4
操作 7: 4 1 3
输出: 5 4 3
操作 8: 1 1 10
操作 9: 4 1 6
输出: 10 5 4 3 7 8

第 15 题图

(1) 请在划线处填入合适的代码。

```
def insert_node(L, head, pos, value):  
    new_node = [value, -1]  
    if pos == 1:  
        new_node[1] = head  
        L.append(new_node)  
        head = len(L)-1  
    else:  
        index = head  
        for j in range(pos - 2):  
            index = L[index][1]  
        new_node[1] = L[index][1]  
        L.append(new_node)
```

```
return head
```

```
def reverse_segment(L, head, start, end):  
    if start == end:  
        return  
    pre = -1  
    cur = head  
    for j in range(start - 1):  
        pre = cur  
        cur = L[cur][1]  
    start_index = cur  
    for j in range(end - start):  
        cur = L[cur][1]  
    end_index = cur  
    next_index = L[end_index][1]  
    ne = next_index  
    c = start_index  
    while c != next_index:  
        # 加框处  
    if pre != -1:  
        L[pre][1] = ne  
    else:  
        head = ne  
    return head
```

```
def delete_segment(L, head, start, end):  
    pre = -1  
    cur = head  
    for j in range(start - 1):  
        pre = cur  
        cur = L[cur][1]  
    for j in range(end - start):  
        cur = L[cur][1]  
    ②  
    if pre != -1:  
        L[pre][1] = next_index  
    else:  
        head = next_index  
    return head
```

```
def print_segment(L, head, start, end):  
    result = []  
    index = head  
    for j in range(start - 1):  
        index = L[index][1]  
    for j in range(end - start + 1):
```

```

        result.append(str(L[index][0]))
    ③
    print("输出: "+" ".join(result)) # 将列表 result 的元素顺序连接成字符串并输出

L = [[i, i] for i in range(1,21)]
L[-1][1] = -1
h = 0
m = int(input("请输入操作总数m: "))
for k in range(m):
    print("操作"+str(k+1)+" : ",end="")
    a, b, c = map(int, input().split()) # 变量 a、b、c 分别存储某个操作需要的三个整
数
    if a == 1:
        h = insert_node(L,h,b,c)
    elif a == 2:
        h = reverse_segment(L,h,b,c)
    elif a == 3:
        h = delete_segment(L,h,b,c)
    elif a == 4:
        print_segment(L,h,b,c)

```

(2) 加框处代码应该为 (单选) __

- A.

```

L[c][1] = ne
ne = c
c = L[c][1]

```

- B.

```

c = L[c][1]
L[c][1] = ne
ne = c

```

- C.

```

t = L[c][1]
ne = c
L[c][1] = ne
c = t

```

- D.

```
t = L[c][1]
L[c][1] = ne
ne = c
c = t
```

答案

正确答案： (1) ① `L[index][1]=len(L)-1` ; ② `next_index=L[cur][1]` ; ③ `index=L[index][1]` ; (2) D

解析

插入节点时，新节点已追加到列表末尾，其下标为 `len(L)-1`，需要让前驱节点指向这个新节点。删除区间时，循环后 `cur` 指向待删区间的最后一个节点，需要用 `next_index=L[cur][1]` 保存后继。输出区间时，每输出一个节点后应沿指针移动到下一个节点。反转链表片段时必须先保存当前节点原后继 `t=L[c][1]`，再把当前节点指向已反转部分的头 `ne`，更新 `ne=c`，最后令 `c=t` 继续处理，因此选 D。

B2：有序区间链表插入与合并

出处：[202606-九校-高二-期末-15](#)。（来源：[202606-九校-高二-期末.pdf](#)；难度：困难）

15. 该系统运行过程中会自动找出所有连续超标的时段（温度超过 27°C ），并将它们按开始时段升序排列整理成一份清单。用户可以根据实际情况修正清单中的数据，如：传感器出错导致误报的记录可以手动删除，因为网络问题漏报的超标时段可以手动补录，确保记录准确。每天凌晨 0:00，系统还会自动统计前一天每个超标时段的持续时长（分钟），并按持续时长降序排序，持续时长相同时按时间先后升序排序，并输出持续时长前 3 条记录，供用户第二天优先检查处理。

例如：某日自动识别出的连续超标时段情况如下，按开始时段升序记录：

序号	时段范围	持续时长（分钟）	说明
1	[48, 60)	60	凌晨长时间超标
2	[96, 102)	30	上午短时超标
3	[156, 168)	60	午后高温
4	[200, 204)	20	傍晚短时超标
5	[240, 252)	60	晚间超标

(1) 用户核查时发现以下问题：

① [96, 102) 时段是由传感器瞬时故障引起，需手动删除；

② 因硬件故障，实际存在某一温度传感器 [168, 172) 超标未记录，需手动补充。

完成数据修正后，该日的高温预警记录最长持续时长为 ▲ 分钟（填数字）。

(2) 定义如下函数 bubble，用于前一天超标持续时长的排序。请在划线处填入合适代码。

```
def bubble(data):
    n = len(data)
    for i in range(n - 1):
        for j in range(0, n - i - 1):
            if data[j][1] <= data[j + 1][1]:
                data[j], data[j + 1] = data[j + 1], data[j]
```

程序段加框处代码有误，需修改为 ▲。

(3) 实现上述功能的部分 Python 程序如下，请在划线处填入合适代码。

```
def ins(head, st, ed):
    # ins 函数保证插入后链表节点始终按开始时段升序排列
    idx = len(dr)
    dr.append([st, ed, -1]) # dr 追加一个新元素 [st, ed, -1]
    if ① :
        dr[idx][2] = head
        return idx
    q = head
    p = dr[head][2]
    while p != -1 and dr[p][0] < st:
        q = p
        p = dr[p][2]
    ②
    dr[q][2] = idx
    return head

def rm(head, st, ed):
    # 删除开始时段为 st 的节点，返回新头节点，若无节点，返回 -1，代码略

def merge(head):
    # 遍历链表，合并所有相邻或重叠的区间，返回新头节点，代码略

# 以下是主程序部分
# 从昨日 sg 中提取超标时段数据，合并连续段后保存为 dr 中
# 例: dr = [[48,60,1],[96,102,2],[156,168,3],[200,204,4],[240,252,-1]]
head = 0
while True:
    data = []
```

```

p = head
while p != -1:
    start = dr[p][0]
    sc =      ③
    data.append([start, sc])
    p = dr[p][2]
bubble(data)
print("昨日高温预警记录: ")
for i in range(min(3, len(data))):
    print("开始时段:", data[i][0], ", 时长:", data[i][1], "分钟")
c = input("\n 请输入指令: 1. 删除 2. 补录 3. 退出")
if c == "1":
    # 删除操作, 代码略
elif c == "2":
    st = int(input("开始时段: "))
    ed = int(input("结束时段: "))
    if st < ed:
        head = ins(head, st, ed)
        head = merge(head)
elif c == "3":
    break

```

答案

正确答案: (1) 80; (2) `data[j][1] < data[j+1][1]`; (3) ① `head == -1 or st < dr[head][0]`; ② `dr[idx][2] = p`; ③ `(dr[p][1] - dr[p][0]) * 5`

解析

删除 [96,102) 后, 再补入 [168,172), 该区间与 [156,168) 相邻, 合并为 [156,172), 持续 16 个时段, 即 80 分钟。排序要求持续时长降序, 相同时保持原有时间先后, 因此只有后一项时长更大时才交换。链表插入时, 空表或新开始时段小于头节点开始时段要插到表头; 找到插入位置后新节点后继应接原来的 p, 再让 q 指向新节点; 持续时长由结束时段减开始时段后乘以 5 分钟得到。

B3: 座位可选区间链表维护

出处: [202604-宁波-高三-二模-15](#)。(来源: [202604-宁波-高三-二模.pdf](#); 难度: 较难)

15. 某影院推出“智慧观影”系统, 该平台具有以下功能: a) 观众进入平台后, 可查看影片综合评分排行榜。将本月所有影片按综合评分从高到低排序, 并输出排行榜名单。b) 观众选择心仪影片后, 系统根据实时状态更新座位信息, 为观众呈现可选座位区间。初始状态所有座位均可选, 当有观众购买某座位时, 系统将该座位从可选区间中移除; 当有观众退票时, 系统将该座位合并到相邻的可选区间中, 或创建新区间。如某排共 10 个座位, 购买座位 3、4、9 后, 该排可选区间为 [1,2]、[5,8]、[10,10]; 若座位 4 退票: 与区间 [5,8] 左邻接, 此时可选

区间为 [1,2]、[4,8]、[10,10]; 若座位 3 退票: 与区间 [1,2] 右邻接, 且与 [4,8] 左邻接, 三个区间合并为 [1,8]、[10,10]。

(1) 某影院每排共 10 个座位, 初始状态所有座位均可选。09:15 前座位状态变化记录如下所示 (只记录发生变化的座位) 。

时间	座位号	排数	状态
08:37	3	5	购票
08:50	5	5	购票
09:00	3	5	退票
09:12	8	5	购票

请根据记录, 写出 09:15 时 5 排的可选座位区间 (用 [起始座位号, 终止座位号] 形式表示) _____。

(2) 定义如下函数, 参数 b 列表中每个元素包含 4 个数据项, 依次为电影名称、上映时间、影片类型、综合得分, 参数 n 表示列表 b 的长度, 如 `b[0]=["天才游戏", "2026-04-04", "悬疑/犯罪", 8.3]` 。

```
def fsort(b, n):
    if n < 2:
        return b
    for i in range(n - 1):
        if b[i][3] < b[i + 1][3]:
            b[i], b[i + 1] = b[i + 1], b[i]
    return fsort(b, n - 1)
```

下列关于该函数的说法, 正确的是_____ (单选) 。

- A. 该函数功能为将列表 b 中的元素按综合得分升序排序
- B. 该函数功能为将列表 b 中的元素按上映时间降序排序
- C. 加框处代码修改为 `(n-2, -1, -1)`, 不改变函数功能
- D. 将语句 1 处代码中 `n<2` 改为 `n<=0`, 不改变函数功能

(3) 系统可以实时处理购/退票信息, 为观众呈现当前可选座位区间, 其中处理购票功能的函数如下:

```
def order_seat(seat, row, lst):
    pre = 0
    p = lst[pre][4]
    while p != -1 and lst[p][0] != row:
```



```

pre = p
p = lst[p][4]
while p != -1 and lst[p][0] == row:
    if seat == lst[p][1] and lst[p][3] == 1:
        -----①-----
    elif seat == lst[p][1] and lst[p][3] > 1:
        lst[p][1] = seat + 1
        lst[p][3] -= 1
    elif -----②-----:
        lst[p][2] = seat - 1
        lst[p][3] -= 1
    elif seat > lst[p][1] and seat < lst[p][2]:
        right = lst[p][2]
        lst[p][2] = seat - 1
        -----③-----
        lst.append([row, seat + 1, right, right - seat, lst[p][4]])
        lst[p][4] = len(lst) - 1
pre = p
p = lst[p][4]

```

请在划线处填入合适的代码。

答案

正确答案： (1) [1,4], [6,7], [9,10]; (2) D; (3) ① `lst[pre][4] = lst[p][4]`; ② `seat == lst[p][2] and lst[p][3] > 1`; ③ `lst[p][3] = seat - lst[p][1]`, 或 `lst[p][3] = lst[p][2] - lst[p][1] + 1`。

解析

购票会从可选区间中移除座位，退票会与相邻可选区间合并。第 5 排按记录更新后剩余区间为 [1,4]、[6,7]、[9,10]。fsort 按综合得分降序递归冒泡， $n < 2$ 改为 $n \leq 0$ 仍能完成。链表删除单座区间时应跳过当前结点；若购买的是区间右端点且区间长度大于 1，则右边界左移并更新可选数。

B4：时间段链表插入与统计

出处：[202605-县域教研-高三-月考-15](#)。（来源：[202605-县域教研-高三-月考.pdf](#)；难度：困难）

15. 为准备某次拔河比赛，每个班级内部比赛前打算训练一下。为此，学校打算给每个班级（每个班有班级编号）在训练时准备一根绳子，用于某天内的某个时间段进行拔河训练。事先，每个班根据自己班级的情况向学校报送了各班级训练拔河比赛的时间段（包含开始时间和结束时间），如第 15 题图所示为报送的部分班级数据。

班级编号	101	201	104	301	202
开始时间	07:30	08:20	08:30	09:40	08:50
结束时间	10:30	09:30	09:20	11:30	09:55

第 15 题图

根据各班级上报的训练时间段，学校为节约开支，打算在满足各班级训练时间需求的情况下采购最少数量的绳子。在训练的过程中，若一个班级训练结束，从结束时间的下一分钟开始，这根绳子即可被另外一个班级使用，如某班 8:30 结束，则 8:31 开始这根绳子可以被后面时段训练的班级使用。请回答下列问题：

(1) 在不考虑其他班级的情况下，若要满足第 15 题图所示班级的训练时间需求，学校需要采购的绳子最少为 __ 根。

(2) 定义如下函数 `d_sort(data,k)`，该函数的功能是对列表 `data` 中的元素按每个元素索引为 `k` 的数据项进行排序。

```
def d_sort(data, k):
    i = p = q = len(data) - 1
    while i > 1 and q != 0:
        j = q = 0
        while j < p:
            if data[j][k] > data[j + 1][k]:
                data[j], data[j + 1] = data[j + 1], data[j]
                q = j + 1
            j += 1
        p = q - 1
        i -= 1
    return data
```

调用该函数，请回答下列问题。

① 若程序中加框处语句修改为 `p = q`，该函数的运行结果 __（单选，填字母 A 或 B：A 表示会，B 表示不会）发生变化。

② 若 `data` 为 `[[101,450,630],[201,500,570],[104,510,560],[301,580,690],[202,530,595]]`，则调用函数 `d_sort(data,1)` 对 `data` 进行排序的过程中，`p` 的值最小为 __。

(3) 根据各班级拔河训练时段计算最少绳子采购数量的 Python 程序如下：
请在划线处填入合适的代码。

```
def d_sort(data, k):
    # 函数的功能是根据 data 元素的第 k 个数据项进行升序排序
    # 代码略
```

```
def t_to_m(tstr):
    # 函数的功能是将 "HH:MM" 格式的时间转换为分钟数
    # 代码略
```

```
def ins(lnk, head, x, y): # 插入
    p = head
    while p != -1 and lnk[p][0] <= x: # 找到插入位置
        q = p
        p = lnk[p][2]
    if _____①_____:
        lnk.append([x, y, head])
        head = len(lnk) - 1
    else:
        lnk.append([x, y, p])
        lnk[q][2] = len(lnk) - 1
    return head
```

```
def proc(data, n):
    lnk = [] # 构造一个空链表
    data = d_sort(data, 1)
    for i in range(n):
        data[i].append(0)
    cnt = 1 # 采购的绳子从 1 开始编号
    -----②-----
    lnk.append([data[0][2], data[0][3], -1])
    head = 0
    for i in range(1, n):
        if data[i][1] > lnk[head][0]:
            data[i][3] = lnk[head][1]
        else:
            cnt += 1
            data[i][3] = cnt
    -----③-----
    return cnt, data
```

```
"""
```

读取班级数量 `n`;

读取各班级拔河训练的时间段信息，存入列表 `data`，每个元素包含班级编号、训练开始时间、结束时间，

如 `data=[[101,450,630],[201,500,570],[104,510,560],[301,580,690],[202,530,595]]`

代码略

```
"""
```

```
ans_cnt, ans_lst = proc(data, n)
print(ans_cnt) # 输出最少需要采购的绳子数量
ans_lst = d_sort(ans_lst, 0) # 根据班级编号进行排序
for i in range(len(ans_lst)):
    print(ans_lst[i][0], ans_lst[i][3]) # 输出各班级训练用的绳子编号
```

答案

正确答案： (1) 4; (2) ① B; ② -1; (3) ① `p == head`; ② `data[0][3] = cnt`; ③ `head = ins(lnk, head, data[i][2], data[i][3])`

解析

根据参考答案中的评分点填写。程序题复习时应重点跟踪变量含义、循环边界和空位所在语句的上下文。

B5：自然子序列识别与链表合并

出处：[202512-精诚联盟-高三-月考-15](#)。（来源：[202512-精诚联盟-高三-月考.pdf](#)；难度：困难）

某数据序列data 中的元素均为正整数。现在要对data 进行处理，处理过程分“去重”“识别”和“合并”三步。对data“去重”处理通过给定的dedup 函数实现。“识别”指在一组数据中先识别出已升序的自然子序列, 如data 为[3,1,6,8,7,5]，可识别出3 个，分别是[3,6,8],[1,7],[5]。在“合并”过程中，算法会优先合并那些长度较短的子序列，最终得到一个完全有序的完整序列。

(1) 若data 为[5,1,6,3,8,7,4,9]，则可依次识别出的自然子序列为：▲ 个。

(2) “去重”处理的dedup 函数如下，请在划线处填入合适的代码。

```
def dedup(a):
    i = 0; n = len(a)
    while i < n:
        r = i + 1
        for j in range(i + 1, n):
            if _____:
                a[r] = a[j]; r += 1
        n = r; i += 1
    return a[:n]
```

(3) 实现处理功能的部分Python 程序如下，请在划线处填入合适的代码。

```
def sort_data(data):
    k = 1
    while k < len(data):
```

```

tmp = data[k]; j = k - 1
①:
    data[j+1] = data[j]
    j -= 1
data[j+1] = tmp
k += 1

```

```

def merge(i):
    qa = pa = ha = runs[i][0]
    qb = pb = hb = runs[i+1][0]
    while pb != -1:
        while pa != -1 and nodes[pa][1] < nodes[pb][1]:
            qa = pa; pa = nodes[pa][2]
        qb = pb; pb = nodes[pb][2]
        nodes[qb][2] = pa
        if pa == ha:
            ha = hb; qa = qb
        else:
            nodes[qa][2] = qb
    ②
    if ha != -1:
        end = runs[i][1]
    else:
        end = runs[i+1][1]
    return [ha, end, runs[i][2]+runs[i+1][2]]

```

读取数据存入data,进行去重处理, n 为去重后data 数据个数, 代码略

```

nodes = [[0, data[0], -1]]
runs = [[0, 0, 1]]
for i in range(1, n):
    k = 0
    while k < len(runs) and data[i] < nodes[runs[k][1]][1]:
        k += 1
    if k < len(runs):
        ③
        runs[k][1] = i; runs[k][2] += 1
    else:
        runs.append([i, i, 1])
    nodes.append([i, data[i], -1])
i = 0
while i < len(runs)-1:
    sort_data(runs)
    runs.append(merge(i))
    i += 2

```

输出处理后的data 序列, 代码略

答案

正确答案： (1) 3; (2) `a[i] != a[j]`; (3) ① `while j >= 0 and data[j][2] > tmp[2]`; ② `qa = nodes[qa][2]`; ③ `nodes[runs[k][1]][2]=i`

解析

(1) 按自然升序子序列识别, `[5,1,6,3,8,7,4,9]` 可分为 `[5,6,8]`、`[1,3,7]`、`[4,9]`, 共 3 个。(2) 去重时保留与当前基准 `a[i]` 不同的后续元素, 因此条件为 `a[i] != a[j]`。(3) `sort_data(runs)` 按自然子序列长度 `data[][2]` 做插入排序, 所以条件为 `while j >= 0 and data[j][2] > tmp[2]`; 合并链表时, 将 b 链表节点接到 qa 后, 还要让 qa 移到刚插入的节点, 即 `qa = nodes[qa][2]`; 识别自然子序列时, 需要把当前子序列原尾节点的指针指向新节点 i, 即 `nodes[runs[k][1]][2]=i`。

B6: 连续格子整理与链表插入

出处: [202604-9+1高中联盟-高二-期中-15](#)。(来源: [202604-9+1高中联盟-高二-期中.pdf](#); 难度: 困难)

15. 某货架有多个格子组成, 每个格子只能放置一个物品, 格子从 1 开始编号。现已有 n 个物品放置在格子中, 管理员从小号到大号顺序记录了物品所在的格子编号。现要对记录信息进行整理, 格式为“[起始格子编号, 连续放置货物的格子数量]”, 如有 7 个物品放置的格子编号依次为“1,2,3,8”, 则整理后的结果为“[1, 3], [8, 1]”, 第 1 项表示物品放置的起始格子编号, 第 2 项表示连续放置货物的格子数量。程序运行界面如第 15 题图所示。请回答下列问题。

物品放置信息为: [1, 2, 3, 4, 7, 8, 9]
整理后的信息为: [[1, 4], [7, 3]]

第 15 题图

(1) 若 8 个物品的放置格子的编号依次为“2,3,4,7,8,9,15,16”, 则整理后的结果为 __▲__。

(2) 编写 `arrange(lst)` 函数, 功能为将 `lst` 中的物品放置信息按题目要求进行整理。请在程序划线填入合适的代码。

```
def arrange(lst):
    start = lst[0]
    cnt = 1
    for num in lst[1:]:
        if ____▲____:
            cnt += 1
        else:
            lnk.append([start, cnt])
            start = num
```

```

        cnt = 1
    lnk.append([start, cnt])

```

(3) 管理员进行一轮物品取放操作后，需要对之前整理后的信息进行再次整理，如某一轮的取放操作为 [["in",2], ["in",10], ["out",15], ["in",19], ["in",20]]，第 1 项为 "in" 或 "out"，"in" 表示放置物品，"out" 表示取出物品，第 2 项表示放置或取出物品的格子编号，且保证本轮取放操作中放置物品的格子编号之前未被占用、取出物品的格子中之前已放置物品。实现再次整理过程的部分 Python 程序如下，请在划线处填入合适的代码。

```

def solve(d, h):
    p = q = h
    if d[0] == "in": # 放置物品
        if lnk[p][0] > d[1] + 1:
            lnk.append([d[1], 1, q])
            h = len(lnk) - 1
            return h
        while q != -1 and ____①____:
            p = q
            q = lnk[q][2]
            if d[1] == lnk[p][0] + lnk[p][1] and d[1] + 1 == lnk[q][0]:
                lnk[p][1] = lnk[p][1] + lnk[q][1] + 1
                ____②____
            elif d[1] == lnk[p][0] + lnk[p][1]:
                lnk[p][1] += 1
            elif d[1] + 1 == lnk[q][0]:
                lnk[q][0] -= 1
                ____③____
            else:
                lnk.append([d[1], 1, q])
                lnk[p][2] = len(lnk) - 1
    elif d[0] == "out":
        # 取出物品，代码略

```

将放置物品的格子编号数据存储在列表 a 中，形式如 [1, 2, 3, 4, 7, 8, 9]

```

lnk = []
n = len(a)
arrange(a)
print("物品放置信息为: ", a)
print("整理后的信息为: ", lnk)
for i in range(1, len(lnk)):
    lnk[i-1].append(i)
lnk[-1].append(-1)
# 某一轮的取放操作数据存放在列表 info 中，形式如 [ ["in",5], ['out',15], ['in',11]]
m = len(info)
head = 0

```

```

for i in range(m):
    head = solve(info[i], head)
p = head
while p != -1:
    print(lnk[p][0:2], end=" ", " ")
    p = lnk[p][2]

```

答案

正确答案： (1) [2,3],[7,3],[15,2] ; (2) num == start + cnt ; (3) ① $d[1] \geq \text{lnk}[q][0] + \text{lnk}[q][1]$ 或 $d[1] > \text{lnk}[q][0] + \text{lnk}[q][1] - 1$; ② $\text{lnk}[p][2] = \text{lnk}[q][2]$; ③ $\text{lnk}[q][1] += 1$

解析

第 (1) 题按连续格子分段即可。arrange 中当当前编号等于起始编号加连续数量时，说明仍连续。第 (3) 题中 lnk 用 [起点, 数量, 后继] 模拟链表；①用于找到插入位置，②用于合并前后两个连续区间后跳过 q 节点，③用于把新区间扩展到 q 节点左侧并增加长度。

B7：货架装载与链表连接

出处：[202604-嘉兴-高三-二模-15](#)。（来源：[202604-嘉兴-高三-二模.pdf](#)；难度：困难）

现有 n 种编号为 $1 \sim n$ 的商品，每种数量若干，需放入货架。每个货架最多放 m 个商品，货架编号从 1 开始且数量充足。入库规则如下：

第一轮：按商品编号从小到大的顺序处理每一种商品。如果某种商品的数量 $\geq m$ 个，就把 m 个商品放满一个货架，如果还能再放满一个货架，就继续放。当剩余数量不足 m 个时暂不放置，留到第二轮处理。

第二轮：按“剩余商品数量从大到小”排序（数量相同则编号小的优先），依次放置剩余商品：①优先放入已有货架中：剩余空位 $\geq$ 该商品剩余数量，且空位最少、编号最小的货架；②若无满足条件的已有货架，则放入新货架。

例：现有 3 种编号的商品需入库，每个货架最多能放 15 个商品，各编号商品数量及放置过程如第 15 题图所示。

编号	数量
1	19
2	10
3	20

编号	数量	编号
1	4	1
2	10	
3	5	2

编号	数量	编号
2	10	3
3	5	2, 3
1	4	1, 4

第 15 题图

请回答下列问题：

(1) 若仅将第 15 题图中编号 1 的商品数量改为 39，则编号为 3 的商品放置的货架编号依次为 __▲__。（货架编号间用逗号分隔）

(2) 定义如下 `lnkinsert(x, head)` 函数，功能为将商品信息 `x` 插入到链表 `lst` 中，使得链表按商品剩余数量降序排列。`lst` 节点的前两项数据依次为商品编号和剩余数量，第三个数据项为指针。`x` 为列表，其中 `x[0]` 为商品编号，`x[1]` 为商品剩余数量，`head` 为链表头的指针。

为实现上述功能，请在程序划线处填入合适的代码。

```
def lnkinsert(x, head):
    if head == -1:
        lst.append([x[0], x[1], -1])
        head = 0
    elif x[1] > lst[head][1]:
        lst.append([x[0], x[1], head])
        head = len(lst) - 1
    else:
        p = q = head
        while q != -1 and x[1] < lst[q][1]:
            p = q
            q = lst[q][2]
        lst.append([x[0], x[1], q])
        -----▲-----
    return head
```

(3) 实现商品放置功能的部分 Python 程序如下，请在程序中划线处填入合适的代码。

```
def place(goods, m):
    n = len(goods)
    result = [] # 存放各类商品所放置的货架编号
    for i in range(n + 1):
        result.append([])
    rest = [m] * 101 # 假设货架数不超过 100 个
    hjbh = 0
    head = -1
    for i in range(n):
        while goods[i][1] >= m:
            hjbh = hjbh + 1
            result[goods[i][0]].append(hjbh)
            -----①-----
        if goods[i][1] > 0:
            head = lnkinsert(goods[i], head)
    start = hjbh + 1
    p = head
    while p != -1:
```

```

cnt = lst[p][1]
empty = m
for j in range(start, hjbh + 1):
    if ____②____:
        empty = rest[j]
        ans = j
if empty == m:      # 若找不到，则新增一个货架，将其放置在新货架上
    ____③____
    ans = hjbh
result[lst[p][0]].append(ans)
rest[ans] -= lst[p][1]
p = lst[p][2]
return result

```

主程序

列表 goods 存储编号 1 至 n 的商品数据，依次存入 goods[0] 至 goods[n-1] 中；

goods[i] 包含 2 个数据项，goods[i][0]、goods[i][1] 分别存放商品编号及数量。

```
lst = []
```

```
m = int(input("m="))
```

```
goods = [[1,31], [2,17], [3,11], [4,40]]
```

```
print(place(goods, m))
```

答案

正确答案： (1) 3, 4; (2) `lst[p][2] = len(lst) - 1`; (3) ① `goods[i][1] -= m`; ② `rest[j] >= cnt and rest[j] < empty` 或 `empty > rest[j] >= cnt`; ③ `hjbh += 1`

解析

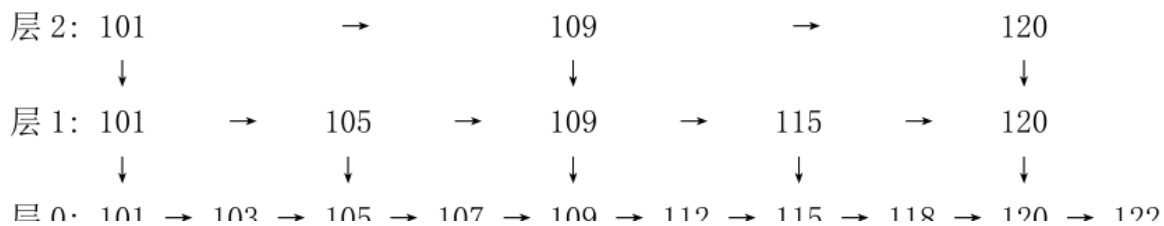
链表插入时，新节点追加到 `lst` 末尾，前驱节点 `p` 的指针应改为新节点下标 `len(lst)-1`。第一轮每放满一个货架，当前商品剩余量减少 `m`。第二轮要在已有货架中找能容纳当前剩余数量且剩余空间最小的货架，因此判断条件为 `rest[j] >= cnt and rest[j] < empty`。找不到合适货架时新增货架，货架编号 `hjbh` 加 1。

B8：跳跃表查找、插入与删除

出处：[202603-新阵地-高三-月考-15](#)。（来源：[202603-新阵地-高三-月考.pdf](#)；难度：困难）

15. 某图书馆需要存储一组按升序排列的图书编号。为提高查找、插入与删除等操作的效率，决定采用“跳跃表”结构进行存储。跳跃表是一种多层链表，其中每一层都是下一层的子集。借助高层链表直接跳过部分底层节点，实现对数据的快速访问。

如给定以下初始数据（已按编号升序存入列表 `books=[101, 103, 105, 107, 109, 112, 115, 118, 120, 122]`），跳跃表的结构设计如下，假设层 `i` 的跳跃步长为 2^i ：



跳跃表的查找规则为：从最高层开始，在当前层向右逐个访问节点，直至遇到下一个节点大于目标值时，下降至下一层继续查找；重复此过程，直至找到目标节点或确认其不存在。

(1) 若需在跳跃表中查找为 107 的图书，从最高层（即第 2 层）开始查找。则查找过程中节点的跳跃顺序为：__▲__（填写节点编号，用 → 连接）。

(2) 定义函数 `build_skip(books, step_list)`，其功能是根据步长列表 `step_list` 构建跳跃表。函数返回一个多层链表结构 `skip`，其中 `skip[i]` 代表第 `i` 层链表的节点列表，每个节点用列表 [编号, 下层索引, 本层下一索引] 表示。请在以下划线处填入恰当的代码：

```
def build_skip(books, step_list):
    n = len(books)
    layers = len(step_list)
    skip = [[] for i in range(layers)]
    for i in range(layers): # 构建每一层链表
        skip[i].append([books[0], -1, -1]) # 添加头节点
        step = step_list[i]
        t = 0
        for j in range(step, n, step):
            node = [books[j], -1, -1]
            skip[i].append(node)
            skip[i][t][2] = len(skip[i]) - 1
            -----①-----
        skip[i][t][2] = -1
    for i in range(layers - 1, 0, -1): # 建立层与层之间的链接
        p1 = 0 # p1 为下层索引
        p2 = 0
        while p2 != -1:
            if skip[i][p2][0] == skip[i-1][p1][0]:
                skip[i][p2][1] = p1
                -----②-----
            p1 = skip[i-1][p1][2]
    return skip
```

(3) 定义函数 `search_skip(skip, target)`，其功能是在跳跃表 `skip` 中查找编号为 `target` 的图书。若找到该图书，则返回 `True`；否则，返回 `False`。请在划线处填入合适的代码：

```
def search_skip(skip, target):
    p = 0
```

```

for i in range(len(skip) - 1, -1, -1):
    q = skip[i][p][2]
    while ____①____:
        p = q
        q = skip[i][p][2]
    if skip[i][p][0] == target:
        return True
    ____②____
return False

```

答案

正确答案： (1) $101 \rightarrow 105 \rightarrow 107$; (2) ① $t = \text{len}(\text{skip}[i]) - 1$ 或 $t += 1$; ② $p2 = \text{skip}[i][p2][2]$; (3) ① $q \neq -1$ and $\text{skip}[i][q][0] \leq \text{target}$; ② $p = \text{skip}[i][p][1]$

解析

构建同层链表时，每追加一个节点后需把 t 更新为新节点索引。建立上下层连接时，当前层匹配成功后 $p2$ 向本层后继移动。查找时只要下一节点存在且编号不超过目标值就向右跳；一层结束后通过下层索引下降。

B9：酒店配送订单链表调度

出处：[202512-Z20名校联盟-高三-月考-15](#)。（来源：[202512-Z20名校联盟-高三-月考-试卷.pdf](#)；难度：困难）

15. 某酒店有 3 个单元楼（1 单元、2 单元、3 单元），每个单元楼有 10 层（1-10 层），配送机器人从地面（算作第 0 层）出发需按订单提交顺序配送外卖，配送规则如下：

- ①优先配送同一单元楼的外卖，凑满 3 件后配送（机器人一次性最多配送三件）；
- ②若同一单元楼订单不足 3 件，且后续 5 分钟内无该单元楼新订单，则直接配送当前订单；
- ③配送时间计算：每层楼电梯运行时间按 1 分钟一层楼计算；抵达楼层后每配送 1 件外卖需额外 3 分钟（如 1 单元 3 层配送 1 件需 6 分钟，如 1 单元 3 层、5 层各 1 件，共需 11 分钟）。

请回答下列问题：

订单编号	提交时间（分钟）	单元楼	楼层
D01	840	1单元	3层
D02	841	2单元	5层
D03	842	1单元	5层
D04	843	3单元	8层
D05	844	1单元	2层
D06	846	2单元	7层
D07	847	1单元	6层

(1) 已知某时段外卖订单按提交顺序排列如下表，提交时间已转换为分钟，例 840 分钟表示 14 点整。初始无待配送订单。根据配送规则，第 1 次 1 单元配送的订单需 ____ 分钟。

(2) ①定义 `queIn` 函数用于实现按单元划分订单。其中，`data` 列表存储订单信息，每个元素初始格式为：【订单编号，提交时间（分钟），单元楼，楼层，指针域】。划线处代码为 ____。

```
que = [[-1, -1], [-1, -1], [-1, -1]]
head = 0
def queIn(head):
    while head < len(data):
        k = data[head][2]
        if que[k-1][1] == -1:
            que[k-1][0] = head
            que[k-1][1] = head
        else:
            data[que[k-1][1]][4] = head
            -----▲-----
            head += 1
```

②定义 `proc` 函数实现：根据各单元待配送订单状态对配送单元进行排序。规则如下：按照各单元待配送订单数量由多到少配送，若数量相同则按首件订单提交时间由先到后配送。实现上述功能的代码如下：

```
def proc(qs, ucnt): #列表ucnt存储各单元待配送订单数量
    unit = [1, 2, 3]
    n = len(ucnt)
    for i in range(1, n):
        for j in range(n - i):
```

```

        if -----^-----:
            ucnt[j], ucnt[j+1] = ucnt[j+1], ucnt[j]
            unit[j], unit[j+1] = unit[j+1], unit[j]
    return unit

```

则划线处正确的代码为 ____（单选，填字母）。

- A. `ucnt[j] > ucnt[j+1] or ucnt[j] == ucnt[j+1] and data[qs[unit[j]-1][0]][1] > data[qs[unit[j+1]-1][0]][1]`
- B. `ucnt[j] > ucnt[j+1] or ucnt[j] == ucnt[j+1] and data[qs[unit[j]-1][0]][1] < data[qs[unit[j+1]-1][0]][1]`
- C. `ucnt[j] < ucnt[j+1] or ucnt[j] == ucnt[j+1] and data[qs[unit[j]-1][0]][1] > data[qs[unit[j+1]-1][0]][1]`
- D. `ucnt[j] < ucnt[j+1] or ucnt[j] == ucnt[j+1] and data[qs[unit[j]-1][0]][1] < data[qs[unit[j+1]-1][0]][1]`

(3) 实现机器人配送功能 Python 程序如下。请在划线处填入合适的代码。

```

def delivery(d, curtime):
    #指定单元楼单次配送
    waittime = 5; total = 0 #统计配送耗时
    p = que[d-1][0]
    temp = [] #存储本次配送的订单索引
    ft = data[p][1]
    while p != -1:
        temp.append(p)
        st = data[p][1]
        wt = st - ft
        if len(temp) >= 3 or wt >= waittime or p == que[d-1][1]:
            maxf = 0
            for i in temp:
                if data[i][3] > maxf:
                    maxf = data[i][3]
            ①
            total = maxf + 3 * len(temp)
            curtime += total
            break
        p = data[p][4]
    return total, curtime

curtime = data[0][1]; queIn(head)
while not(que[0][0] == -1 and que[1][0] == -1 and que[2][0] == -1):
    unit = proc(que, ucnt)
    ②

```

```
total, curtime = delivery(d, curtime)
print("本次配送的单元是: ", d, "共耗时" + str(total) + "分钟")
```

答案

正确答案:

- (1) 14 或 17 (1 分)
- (2) ① `que[k-1][1] = head` (2 分) ② C (2 分)
- (3) ① `que[d-1][0] = data[temp[-1]][4]` 或 `que[d-1][0] = data[p][4]` (2 分)
② `d = unit[0]` 或 `d = unit.pop(0)` (2 分)

解析

(1) 需要根据订单表推算。1 单元第一轮配送凑满 3 件或等 5 分钟。根据题干中的订单表，第一次 1 单元配送耗时 14 (或 17，取决于对规则的具体解读)。

(2) ① 尾插法链表的入队操作：新节点的指针域已设置 (`data[que[k-1][1]][4] = head`)，还需要更新队尾指针 `que[k-1][1] = head`。

② 冒泡排序，按订单数量降序排列 (数量多优先配送)。若 `ucnt[j] < ucnt[j+1]` (后面的更多)，需交换；若数量相同且首件提交时间 $>$ (后面的更早)，也需交换。选 C。

(3) ① 配送完成后，从该单元队列中移除已配送的订单，`que[d-1][0]` 指向 `temp` 最后一个订单的后继 (即剩余队列的新队首)。

② `proc` 返回排序后的单元优先级，`d = unit[0]` 取优先级最高的单元进行配送。

B10：订单流水线链表选择

出处：[202512-余姚中学-高二-月考-15](#)。(来源：[202512-余姚中学-高二-月考.pdf](#)；难度：困难)

15. 某工厂每天会收到预约加工订单，某天的订单数据如第15题图a所示。订单种类分为A、B、C、D四种，不同种类订单的单个价值不同，如第15题图b所示。该工厂共有三条加工订单的流水线，各流水线的加工速度如第15题图c所示。订单价值=订单数量×单个价值，工厂收益是完成加工的各订单的价值总和。为实现收益最大化，工厂引进流水线调度系统，当流水线空闲时，该系统优先选择加工已送达订单中价值最高的订单，若有多条流水线空闲，优先为加工速度最快的流水线安排订单。编写程序，模拟系统的订单分配过程，输出每条流水线加工的订单及该订单加工完成的时间。

订单序号	预计送达时间	数量	种类
1	08:03	32	B
2	13:50	24	D
3	08:03	24	A
4	08:06	40	C
5	08:10	72	C
...	...	...	...

第 15 题图 a

订单种类	单个价值
A	6
B	4
C	2
D	1

第 15 题图 b

流水线	加工速度
1	8件/分钟
2	4件/分钟
3	2件/分钟

第 15 题图 c

(1) 已知1号流水线加工的前3个订单的订单序号是1、3、4，订单详情如第15题图a所示，则订单序号为1的订单加工完成
的时刻为

▲

(填写格式如08:00)。

(2) 模拟加工过程，输出每条流水线加工的订单及该订单加工完成时间的部分Python程序如下，请在划线处填入合适的代码。

```
# 读取数据存储在列表data
# data 中每个元素由订单序号、预计送达时间、数量和种类构成，预计送达时间
# 已转换成对应分钟数，种类已转换成对应价值。如某元素[1, "08:03", 32, "B"]已转
# 换为[1, 483, 32, 4]。所有元素按预计送达时间升序排序
# 若预计送达时间相同则按

# 总价值降序排序
# ， 代码略
n=len(data)
speed=[8, 4, 2]
# 三条流水线的加工速度
line=[0]*len(speed) #记录三条流水线的加工结束时间
finish=[[] for i in range(len(speed))]

for i in range(n):
    data[i].append(-1)
i=0; head=-1
while i<n or head!=-1:
    first=0
    ear_time=line[0]
    for j in range(1, len(line)): #寻找最先加工结束的流水线
        if line[j]<ear_time:
            ear_time=line[j]
            ①
    while i<n and data[i][1]<=ear_time:
        #处理流水线加工结束前到达的订单
        if head==-1:
            head=i
        else:
```



```

p=head
q=p
values=data[i][2]*data[i][3]
while p!=-1 and values<data[p][2]*data[p][3]:
    q=p
    ②
if p==head:
    data[i][4]=p
    head=i
else:
    data[i][4]=p
    data[q][4]=i

i+=1
if head!=-1:
    line[first]=line[first]+data[head][2]/speed[first]
    finish[first].append([data[head][0],line[first]])
    head=data[head][4]
else:
    line[first]=
    ③
    finish[first].append([data[i][0],line[first]])
    i+=1

```

输出每条流水线加工的订单及该订单加工完成的时间，代码略

答案

正确答案： (1) 08:10; (2) ① first=j; ② p=data[p][4]; ③ data[i][1]+data[i][2]/speed[first]

解析

按题设调度规则，流水线空闲时优先处理已送达订单中价值最高的订单；代码空缺分别用于记录最早空闲流水线、沿待加工链表后移，以及无等待订单时按送达时间计算完成时间。

B11：货物堆叠与链表式货柜

出处：[202512-金丽衢十二校-高三-一模-15](#)。（来源：[202512-金丽衢十二校-高三-一模.pdf](#)；难度：困难）

15. 货物堆叠问题。现有 n 件货物，各货物的重量存储在 a 列表中，现需要把货物堆放在各个货柜中，货柜编号从 1 开始，货柜数量充足。

堆放规则如下：（1）新货物需堆放在比它重的货物之上；（2）若多个货柜符合要求，则选择与货柜顶部货物重量差值最小的货柜存储；若差值相同则选择货柜号小的；（3）若所有货柜的顶部货物都比新货物轻，则新开一个货柜存放。

（1）若货物重量分别为 15, 12, 17, 12, 15, 10, 9，则第 1 个货柜存放了多少件货物？ __

(2) 阅读下面代码，完成填空。

```
# 现有 n 件货物，各货物的重量存储在 a 列表中，a[i] 中有 2 个数据项，
# a[i][0]、a[i][1] 分别表示货物编号及货物重量

n = len(a)
for i in range(n):
    a[i].append(-1)

h = [0]
t = [0]          # 第 1 个货物直接放入第 1 个货柜

def sort_dis(op, x):
    for i in range(1, len(op)):
        for j in range(i, 0, -1):
            if abs(a[t[op[j]]][1] - a[x][1]) < abs(a[t[op[j-1]]][1] - a[x]
[1]):
                ①
    return op

for i in range(1, n):
    m = len(t)
    k = 0
    option = []
    while ②:
        if a[t[k]][1] > a[i][1]:
            option.append(k)
            k += 1
    if ③:
        h.append(i)
        t.append(i)
    else:
        option = sort_dis(option, i)
        k = option[0]
        ④
        t[k] = i

print("堆放所有货物所需的货柜数为:", len(h))
for i in range(len(h)):
    p = h[i]
    print(i + 1, "号货柜堆放情况为:", end="")
    while p != -1:
        if a[p][2] != -1:
            print(a[p][0], "->", end="")
        else:
```

```
print(a[p][0])
p = a[p][2]
```

答案

正确答案： (1) 4; (2) ① `op[j-1], op[j] = op[j], op[j-1]`; ② `k < m`; ③ `len(option) == 0` 或 `option == []`; ④ `a[t[k]][2] = i`

解析

第 1 个货柜依次可放入 15、12、10、9，共 4 件。程序中 `h`、`t` 分别记录各货柜链表的头、尾，`a[i][2]` 记录同一货柜中下一件货物的下标。`sort_dis` 根据顶部货物与新货物的重量差对候选货柜排序，因此①处是交换候选货柜编号；②遍历所有货柜；③无候选货柜则新开货柜；④把原货柜尾节点指向新货物。

B12：超级计算机优先级队列链表

出处：[202603-金丽衢-高三-月考-15](#)。（来源：[202603-金丽衢-高三-月考.pdf](#)；难度：困难）

计算机学院有一台超级计算机，对校内本科生、研究生和教师开放，每种身份对应不同优先级，教师为 0 级，研究生为 1 级，本科生为 2 级，优先级数值越小优先级越高。高优先级别的申请可抢占低优先级别的申请，同优先级别按使用时间早的优先安排，优先级别相同且使用时间也相同则按提交申请时间安排使用。

现有该计算机的使用申请表，包含申请账号，优先级，使用时间，使用时长和指针 5 个数据项，申请数据已经按提交申请时间作升序排序，使用时间为无序，使用时间已经转换为分钟格式。请根据申请时间的数据排出该超级计算机实际使用的时段表，便于使用者合理安排使用时间。

(1) 若申请表数据为 `[['a', 2, 480, 20, -1], ['b', 0, 510, 15, -1], ['c', 0, 490, 30, -1], ['d', 1, 550, 20, -1], ['e', 2, 480, 5, -1], ['f', 1, 520, 6, -1]]`，则账号为 'a' 的用户用了多少个连续时段才完成其任务？__

(2) 实现上述功能的 Python 程序如下，请在划线处填入合适代码。

```
data = [['a', 2, 480, 20, -1], ['b', 0, 510, 15, -1], ['c', 0, 490, 30, -1],
        ['d', 1, 550, 20, -1], ['e', 2, 480, 5, -1], ['f', 1, 520, 6, -1]]
head = [-1] * 3
t = [[] for i in range(len(data))]
for i in range(len(data)):
    rank = data[i][1]
    time = data[i][2]
    if head[rank] == -1:
        head[rank] = i
    else:
        p = q = head[rank]
```

```

while p != -1 and time >= data[p][2]:
    q = p
    p = data[p][-1]
if p == head[rank]:
    data[i][-1] = p
    ----①----
else:
    data[q][-1] = i
    data[i][-1] = p

clock = 480 # 实验室开放时间
while head != [-1,-1,-1]: # 安排机器使用
    for i in range(3):
        if head[i] != -1 and data[head[i]][2] <= clock and data[head[i]][3] >
0:
            ----②----
            data[head[i]].append(clock)
            if data[head[i]][3] == 0:
                ----③----
                break
        clock += 1

for i in range(len(data)): # 合并连续时间存储在 t 列表中
    t[i].append(data[i][0])
    st = ed = data[i][5]
    for j in range(6,len(data[i])):
        if ----④----:
            ed = data[i][j]
        else:
            t[i].append([st,ed])
            st = ed = data[i][j]
    t[i].append([st,ed])
print(t)

```

答案

正确答案： (1) 3; (2) ① head[rank]=i ; ② data[head[i]][3]-=1 ; ③ head[i]=data[head[i]][4] ; ④ data[i][j]==data[i][j-1]+1

解析

同一优先级内部用链表按使用时间排序，若新节点应插到表头，则更新 head[rank]=i。调度时每分钟减少当前任务剩余时长，因此②为 data[head[i]][3]-=1；任务完成后表头移到后继节点，即③。合并连续时间段时，若当前分钟等于上一分钟加 1，则属于同一连续时段。账号 a 因被更高优先级任务抢占，最终分成 3 个连续时段完成。

B13：购物清单链表删除与统计

出处：[202604-湖衢丽-高三-期中-15](#)。（来源：[202604-湖衢丽-高三-期中.pdf](#)；难度：困难）

某仓库对物品实施入库与出库管理，每件物品对应唯一正整数编号。物品入库时，需将本次入库物品与库中已有物品重新整理，将编号连续的物品归为一堆，编号不连续的物品单独成堆，并根据各堆的物品数量升序排序。物品出库时，若当前待出库数量为 R ，出库过程按以下步骤执行：

- ① 全大于的情况：各堆的物品数量均大于 R ，则从数量最小的堆中取出 R 个物品出库，过程结束。
- ② 不大于的情况：若某堆的物品数量恰好等于 R ，则将该堆全部出库，过程结束；否则从所有物品数量小于 R 的堆中，选取物品数量最大的一个（即最接近 R 的），将该堆物品全部出库，并更新 R （ $R=R-\text{该堆的物品数量}$ ），返回步骤①继续处理。请回答下列问题：

(1) 已知该仓库前 10 个入库的物品编号依次为 3,6,7,1,2,11,5,10,14,13，且中途没有出库操作，则整理后仓库中的物品将分为 __（填数字）堆。

(2) 定义函数 $\text{proc}(p,m)$ ，用于实现将 m 指向的节点根据物品数量有序插入到链表 data 中。链表 data 中每个节点表示一个物品堆，由起始编号、终止编号、指针区域三项组成，已按各堆的物品数量升序排序。

```
def cnt(t):  
    # 计算 t 指向的堆的物品数量  
    return data[t][1]-data[t][0]+1  
  
def proc(p,m):  
    k=data[p][2]  
    while k!=-1 and cnt(k)<cnt(m):  
        p=k  
        k=data[k][2]  
    data[p][2]=m  
    data[m][2]=k
```

若 data 为 $[[1,1,4],[3,3,0],[8,8,1],[5,6,2],[10,15,-1]]$ ，执行语句 $\text{proc}(2,3)$ 后， $\text{data}[3][2]$ 的值为 __。

(3) 模拟仓库处理入库和出库操作的 Python 程序如下，请在划线处填入合适的代码。

```
def stockin(head,id):  
    # 将编号 id 的物品入库，返回链表头指针 head，代码略  
  
def stockout(head,n):  
    res=[]  
    while n!=0:  
        if cnt(head)>n:
```

```

# 从 head 指向的堆取 n 个物品，将其编号加入列表 res，代码略
data[head][0]+=n
n=0
else:
    q=-1
    p=head
    while data[p][2]!=-1 and ____①____:
        q=p
        p=data[p][2]
    n=n-cnt(p)
    # 取出 p 指向的堆中全部物品，将其编号加入列表 res，代码略
    if p==head:
        ____②____
    else:
        data[q][2]=data[p][2]
return res,head

```

获取仓库所有操作数据，存储在列表 `items` 中，每个元素包含两个数据项，`items[i][0]` 为 "in" 时表示入库操作，入库物品编号为 `items[i][1]`；`items[i][0]` 为 "out" 时表示出库操作，出库物品数量为 `items[i][1]`，代码略。

```

data=[]
head=-1
total=0
for item in items:
    if item[0]=="in":
        head=stockin(head,item[1])
        total+=1
    else:
        if total>=item[1]:
            result,head=stockout(head,item[1])
            ____③____
            # 按要求输出 result 的内容，代码略
        else:
            # 取消本次请求，代码略

```

答案

正确答案： (1) 4； (2) 4； (3) ① `cnt(data[p][2])<=n`； ② `head=data[head][2]`； ③ `total-=item[1]`

解析

(1) 入库编号可整理成连续堆：[1,2,3]、[5,6,7]、[10,11]、[13,14]，共 4 堆。(2) `proc(2,3)` 从节点 2 后查找插入位置。节点 3 的堆大小为 2，沿链表经过大小为 1 的节点 1、

0, 停在大小为 6 的节点 4 前, 因此插入后 `data[3][2]=4`。(3) 出库时要在物品数不超过 `n` 的堆中继续找更接近 `n` 的堆, 所以条件为 `cnt(data[p][2])<=n`; 若删除的是头节点, 需要将 `head` 后移到下一节点; 成功出库后库存总数应减少出库数量, 故 `total-=item[1]`。

C组：队列与栈大题

C1：翻译软件缓存队列

出处：[202406-宁波九校-高二-期末-15](#)。（来源：[202406-宁波九校-高二-期末.pdf](#)；难度：困难）

15. 小宁计划设计并制作一款简易机器翻译软件。对该软件的原理作如下构思：依次将英文单词用对应的中文译义来替代。对每个英文单词，先在内存中查找单词的中文译义，如内存中有则进行替代，若没有，则在外存字典中查找，找到单词的中文译义并替代，同时将单词和译义存入内存。假设内存容量为 `m` 个单元，存储的单词数量不能超过 `m`，若超过则需删除最早进入内存的单词。编写程序：输出待翻译文章的翻译结果，并输出需要去外存字典中查找的次数。

(1) 若 `m=3`，待翻译的英文单词为“love python love life python is lifelong love of coders”，则从外存中查找的次数为 __ 次。

(2) 定义如下 `search_disk(key)` 函数，`key` 为待查找的英文单词，函数的功能是在外存中查找单词。

```
def search_disk(key):
    l = 0; r = len(disk)-1
    while l <= r:
        m = (l + r) // 2
        if disk[m][0] == key:
            return disk[m][1]
        elif disk[m][0] > key:
            r = m-1
        else:
            l = m+1
    return 'fail'
```

根据以上代码，请回答①和②两个问题。

①加框处的代码 __（填：能 / 否）改成 `if disk[m][0] > key:`。

②若 `disk` 的长度为 1000，则 `while` 循环最多循环的次数为 __ 次。

(3) 实现翻译并输出结果和外存查找次数的部分代码如下，请在程序划线处填入合适的代码。

```

def search_memory(key):
    p = ①
    while p != -1:
        if memory[p][0] == key:
            return memory[p][1]
        p = memory[p][2]
    return 'fail'

def ins_memory(key, val, m):
    global qhead, qtail # qhead, qtail 为全局变量
    if qtail - qhead >= m:
        for i in range(26):
            if head[i] == qhead:
                ②
                break
        qhead += 1
    memory[qtail] = [key, val, -1]
    num = ord(key[0]) - 97
    if head[num] == -1:
        head[num] = qtail
    else:
        p = head[num]
        while memory[p][2] != -1:
            p = memory[p][2]
        memory[p][2] = qtail
    qtail += 1

```

"""

外存中的单词数据存入字典 **disk** 中。**disk[i]** 包含 2 个数据项，分别为英文单词和中文译义，且按单词从小到大的顺序排列。格式如下 **disk**=[['like', '喜欢'], ['long', '长的'], ['lovely', '可爱的'], ['tail', '尾巴'], ['tall', '高的'].....]

"""

```

m = 10 # 内存容量大小
memory = [0] * 1000
head = [-1] * 26
qhead = qtail = 0
words = input("输入待翻译的英文文章，单词之间以空格分隔").lower().split()
ans = ""
tot = 0
for word in words:
    res = search_memory(word)
    if res == 'fail':
        res = search_disk(word)
        tot += 1
        ③
    ans += res + " "

```



```
print("外存访问总次数为: ",tot)
print("查询结果为: ",ans)
```

答案

正确答案： (1) 8； (2) ①能； ②10； (3) ① `head[ord(key[0])-97]`； ② `head[i]=memory[head[i]][2]` 或 `head[i]=memory[qhead][2]`； ③ `ins_memory(word,res,m)`

解析

第 (1) 小题按容量为 3 的先进先出内存模拟：love、python、life、is、lifelong、love、of、coders 需要访问外存，共 8 次。search_disk 中若已经判断相等并返回，后续比较可以直接写成新的 if，逻辑不变；长度为 1000 的有序表二分查找最多比较 10 次。内存按单词首字母分链存储，查找入口应为 `head[ord(key[0])-97]`。当容量满时，qhead 指向最早进入内存的单词，需要把对应首字母链表头改成该节点的后继。外存查找成功后，应调用 `ins_memory(word,res,m)` 把新单词和译义加入内存。

C2：培训项目录取队列

出处：[202506-杭州-高二-期末-15](#)。（来源：[202506-杭州-高二-期末.pdf](#)；难度：困难）

15. 有 n 位员工申请 m 个培训项目。每位员工对项目有志愿顺序；每个项目对员工进行匹配度评估得到具体分值，分值均不相同。每个项目最多录取 cap 位员工，项目可以不满员，员工也可以不被录取。

培训项目录取规则如下：依次处理每位员工，依照其志愿顺序尝试录取。若当前志愿的项目未招满，则拟录取该员工。若已招满，则当前员工与该项目拟录取员工中匹配度最低者进行比较：①若当前员工匹配度更高，则拟录取该员工，匹配度最低者被淘汰并等待下次处理；②若当前员工匹配度更低，则对当前员工的下一志愿项目尝试录取。直到所有员工都已被拟录取或者未被拟录取员工所有志愿均已尝试，录取过程结束。

例如，有 3 位员工（用大写字母表示）和 3 个项目（用正整数表示），每个项目最多录取 2 位员工。各员工的志愿顺序如第 15 题图 a 所示，各项目的员工匹配度如第 15 题图 b 所示。录取过程如下：员工 A 被项目 3 拟录取 → 员工 B 被项目 3 拟录取 → 员工 C 被项目 3 拟录取（员工 B 被淘汰） → 员工 B 被项目 1 拟录取。录取结束，结果为：项目 1 录取 B，项目 2 没有录取员工，项目 3 录取 A 和 C。

员工编号	志愿顺序
A	项目 3, 项目 2, 项目 1
B	项目 3, 项目 1, 项目 2
C	项目 3, 项目 1, 项目 2

第 15 题图 a

项目编号 \ 员工编号	A	B	C
1	71	82	93
2	88	96	90
3	78	69	89

第 15 题图 b

请回答下列问题：

(1) 若员工 B 的志愿顺序修改为“项目 2, 项目 1, 项目 3”，按上述规则进行录取，则员工 B 的录取结果是 ____（单选，填字母：A. 被项目 1 录取 / B. 被项目 2 录取 / C. 未被录取）。

(2) 定义如下 `sel(a, s, stf)` 函数，参数 `a` 表示某项目已拟录取的员工，参数 `s` 表示该项目的员工匹配度，参数 `stf` 表示当前待录取员工。员工编号“A”~“Z”用 0~25 表示。

```
def sel(a, s, stf):
    idx = 0
    for j in range(1, len(a)):
        if s[a[j]] < s[a[idx]]:
            idx = j
    if s[a[idx]] < s[stf]:
        return idx
    else:
        return -1
```

调用 sel 函数, 若 a 值为 [2, 3], s 值为 [90, 80, 85, 70], stf 值为 1, 则函数返回的结果为 ____。(单选, 填字母: A. -1 / B. 0 / C. 1 / D. 2)

(3) 实现录取过程的部分 Python 程序如下, 请在划线处填入合适的代码。

```
# 读取项目对员工的匹配度分值存储到列表 scores, 如[[71, 82, 93], ...];
# 读取员工的志愿顺序存储到列表 pstf, 如[[3, 2, 1], ...];
# 读取项目人数上限值存储到 cap, 代码略。
m, n = len(scores), len(pstf)      # m 个项目, n 位员工 (n<=26)
cur = [0] * n
adm = [[] for i in range(m)]        # 生成包含 m 个空列表的列表
waitlst = [i for i in range(n + 1)] # 生成从 0 到 n 的整数列表
head, tail = 0, n
while ____①____:
    stf = waitlst[head]
    head = (head + 1) % (n + 1)
    for i in range(cur[stf], m):
        p = pstf[stf][i] - 1
        if ____②____:
            adm[p].append(stf)      # 为 adm[p] 追加一个元素 stf
            break
        else:
            ret = sel(adm[p], scores[p], stf)
            if ret != -1:
                waitlst[tail] = adm[p][ret]
                tail = (tail + 1) % (n + 1)
                ____③____
                break
    cur[stf] = i + 1

for i in range(len(adm)):
    print("项目 " + str(i + 1) + " 录取的员工:", end=" ")
    for j in adm[i]:
```

```
print(chr(j + ord("A")), end=" ")
print()
```

答案

正确答案： (1) B; (2) C; (3) ① `head != tail`; ② `len(adm[p]) < cap`; ③ `adm[p][ret] = stf`

解析

- (1) 修改后 B 首先申请项目 2，项目 2 未满员，所以 B 被项目 2 录取。
- (2) `a=[2,3]` 表示项目当前录取员工 2 和 3，其匹配度分别为 85 和 70，最低者在 `a` 中的下标为 1；待录取员工 1 的匹配度为 80，高于 70，所以返回 1。
- (3) `waitlst` 是循环队列，非空条件为 `head != tail`。若项目 `p` 当前录取人数小于 `cap`，可直接追加当前员工。若当前员工替换了原录取员工，应把 `adm[p][ret]` 改为当前员工 `stf`，原员工已先进入等待队列。

C3：快递储物格队列

出处：[202601-宁波九校-高二-期末-15](#)。（来源：[202601-宁波九校-高二-期末.pdf](#)；难度：困难）

15. 快递送达站点时，优先存放在站点内的储物格，由于数量有限，剩余的快递存放在站点外的临时堆放区。储物格存储分本地件区和外地件区。现站点一共有 n 个储物格，使用遵循“先到先得”的原则，即每个快递送达后，如果相应的区（本地/外地）还有空闲的储物格，就存放在储物格中，否则放在临时区。给定 m_1 个本地件和 m_2 个外地件，以及每一个快递件的送达和取走时间，请你负责将 n 个储物格分配给本地件区和外地件区，使存放于储物格的快递数量最多。

在第 15 题图所示的样例中， n 、 m_1 、 m_2 分别为 3、5、4，当给本地件区分配 2 个储物格、外地件区分配 1 个储物格时，存放的快递数量最多。表格中数据“1,5”表示某快递送达时间和取走时间。

储物格分配方案		本地件送达和取走时间					外地件送达和取走时间				存储物格的快递数量
本地件区	外地件区	1,5	3,8	5,10	9,14	13,18	2,11	4,15	7,17	12,16	
0 个	3 个	×	×	×	×	×	√	√	√	√	4
1 个	2 个	√	×	√	×	√	√	√	×	√	6
2 个	1 个	√	√	√	√	√	√	×	×	√	7
3 个	0 个	√	√	√	√	√	×	×	×	×	5

第 15 题图

请回答下列问题：

(1) 第 15 题图所示数据，若将“1,5”改为“1,7”，则存放在储物格的快递数量最多是 ▲。

(2) 实现该功能的 Python 程序如下，请在程序划线处填入合适的代码。

```
def change(head, tail): # 函数实现对队列 q 进行调整，使得队首元素始终最小
    for i in range( ① ):
        if q[i] < q[i - 1]:
            q[i], q[i - 1] = q[i - 1], q[i]

def check(c, m, k):
    res = 0
    head = tail = 0
    for i in range(m):
        while head < tail and ② :
            head += 1
        if tail - head < k:
            ③
            q[tail] = c[i][1]
            tail += 1
            change(head, tail)
    return res

# 读取储物格数量 n，本地件数量 m1，外地件数量 m2
'''
读取本地件数据存入 a 列表，每个元素包含送达和被取走时间 2 个数据项
例如：a = [[1,5], [3,8], [5,10], [9,14], [13,18]]
读取外地件数据存入 b 列表，每个元素包含送达和被取走时间 2 个数据项
例如：b = [[2,11], [4,15], [7,17], [12,16]]
列表 a 和 b 已分别按送达时间升序排序，代码略
'''
q = [0] * 1000
ans = 0
for i in range(0, n + 1): # 枚举所有可能的分配方案
    ans1 = check(a, m1, i)
    ans2 = ④
    if ans1 + ans2 > ans:
        ans = ans1 + ans2
print("存放在专属储物格的快递数量最多为", ans)
```

答案

正确答案： (1) 6; (2) ① tail - 1, head, -1; ② q[head] <= c[i][0]; ③ res += 1 或 res = res + 1; ④ check(b, m2, n - i)

解析

将本地件 [1,5] 改为 [1,7] 后，枚举本地件区与外地件区的储物格分配，最大可存放数量为 6。程序中 q 保存当前占用储物格的取走时间，队首应保持最小；新取走时间放到队尾后，应从 tail-1 向 head+1 方向调整。若 $q[head] \leq c[i][0]$ ，说明队首快递已被取走，可释放储物格。成功存入储物格时计数 $res += 1$ 。枚举本地件分配 i 个格子时，外地件应分配剩余的 $n-i$ 个格子。

C4：厨师订单分配队列

出处：[202603-强基联盟-高三-月考-15](#)。（来源：[202603-强基联盟-高三-月考.pdf](#)；难度：困难）

15. 某餐厅有 n 名厨师（编号为 0 到 n-1），餐厅根据顾客提交的订单安排厨师，每一条订单包含订单编号（按照提交顺序从 1 开始依次编号）、到达时间、所需时间、菜品类型 4 个数据项，每位厨师有擅长的菜品类型，只能烹制擅长类型的菜品。餐厅按以下规则安排厨师：
- 每个订单都会分配给空闲且擅长该菜品类型的最小编号厨师；
 - 如果没有符合条件的空闲厨师，订单将进入等待队列；
 - 当有厨师空闲时，优先处理等待队列中等待时间最长的订单。

例如：假设 $n=3$ ，厨师 0 和厨师 1 均擅长川菜，厨师 2 擅长粤菜，初始时 3 位厨师均为空闲状态，有 6 条订单数据如下表所示：

订单编号	到达时间	所需时间（分钟）	菜品类型
1	18:00	25	川菜
2	18:20	40	粤菜
3	18:30	30	川菜
4	18:35	25	川菜
5	18:40	20	川菜
6	18:40	40	粤菜

则安排结果如下：

- 厨师 0：订单 1（18:00-18:25），订单 3（18:30-19:00），订单 5（19:00-19:20）
- 厨师 1：订单 4（18:35-19:00）
- 厨师 2：订单 2（18:20-19:00），订单 6（19:00-19:40）

请回答以下问题：

(1) 订单数据如题所述，现增加一条订单 [7, "18:30", 40, "川菜"]，该订单会被分配给 __ (选填：厨师 0 / 厨师 1 / 厨师 2)，实际开始时间是 __。

(2) 定义 insert_work() 函数，work 列表中每个元素的数据项依次为结束时间、厨师编号，函数功能为将 [end_time, chef] 插入 work 列表，并保持按结束时间升序，程序中加框处代码有错，请改正：

```
def insert_work(end_time, chef):
    work.append([end_time, chef])
    i = len(work) - 1
    while work[i][0] > work[i-1][0]:
        work[i], work[i-1] = work[i-1], work[i]
        i -= 1
```

(3) 实现餐厅厨师分配调度的 Python 程序如下，请在划线处填写合适的代码。

```
def con1(s):
    # 将"时:分"格式的时间字符串转换成以"分钟"为单位的整数，代码略

def con2(t):
    # 将以分钟为单位的整数转换成"时:分"格式的时间字符串，代码略

def find_chef(tp):
    # 寻找空闲且擅长指定菜品的厨师，返回厨师 ID，找不到返回 -1，代码略

def assign(chef, oid, st):    # 将订单分配给厨师
    dur = orders[oid][2]
    et = st + dur
    orders[oid][1] = con2(st)
    orders[oid].append(con2(et))    # append: 在列表末尾添加元素
    orders[oid].append(-1)
    idle[chef] = False
    insert_work(et, chef)
    if head[chef] == -1:
        head[chef] = oid
    else:
        -----①-----
        tail[chef] = oid

def arrange(orders, n, skills):    # 主调度函数
    global idle, work, wait, head, tail    # global: 声明全局变量
    idle = [True] * n
    work = []    # 工作队列
    wait = []    # 等待队列
    head = [-1] * n
```

```

tail = [-1] * n
i = 0
now = 0
while i < len(orders) or wait or work:
    while i < len(orders) and con1(orders[i][1]) <= now:
        tp = orders[i][3]
        dur = orders[i][2]
        -----②-----
        if chef != -1:
            assign(chef, i, now)
        else:
            wait.append([i, tp, dur])
        i += 1
    new_work = []
    for et, cid in work:
        if et <= now:
            idle[cid] = True
            if wait:
                for k in range(len(wait)):
                    woid, wtp, wdur = wait[k]
                    chef = find_chef(wtp)
                    if chef != -1:
                        assign(chef, woid, now)
                        wait.pop(k)          # pop: 弹出索引 k 处的元素
                        break
            else:
                new_work.append([et, cid])
    work = new_work
    -----③-----
    if i < len(orders):
        now = min(now, con1(orders[i][1]))
    if work:
        now = min(now, work[0][0])
return head

```

"""

读取 `orders` 数据, `orders` 每个元素包含 4 个数据项, 依次为订单编号、到达时间、所需时间、菜品类型, 代码略。

"""

```

n = 3
skills = ["川菜", "川菜", "粤菜"]
heads = arrange(orders, n, skills)
# 输出厨师安排结果, 代码略

```

答案

正确答案： (1) 厨师 1, 18:30; (2) `i > 0 and work[i][0] < work[i-1][0]`; (3) ① `orders[tail[chef]][5] = oid`; ② `chef = find_chef(tp)`; ③ `now += 1`

解析

增加订单 7 后, 18:30 时厨师 0 已安排订单 3, 厨师 1 空闲且擅长川菜, 因此订单 7 分配给厨师 1 并立即开始。insert_work() 要保持结束时间升序, 应在 `i > 0` 且新元素结束时间小于前一项时交换。assign() 中新订单需要接到该厨师原订单链尾部, 所以修改原尾订单的后继指针; 主调度中应根据菜品类型寻找合适厨师, 并在循环末尾推进当前时间。

C5: 码头车辆装载队列

出处: [202605-镇海中学-高三-月考-15](#)。(来源: [202605-镇海中学-高三-月考.pdf](#); 难度: 困难)

某码头负责装载车辆, 每辆车有唯一的正整数编号。现因天气原因该码头仅开放一个装载点, 供车辆依次上船。装载规则如下:

- I. 每过一个单位时间, 有一辆车到达并加入排队队伍, 同时有一辆车正在装载。
- II. 为保证后续分类方便, 同一品牌的车辆必须在队伍中连续排列。若队伍中同时有品牌 0 和品牌 1 的车辆, 且当前正在装载的是品牌 1 的车, 则接下来会优先让所有品牌 1 的车完成装载, 再处理品牌 0 的车。
- III. 当一辆新车到达时, 如果队伍中已有同品牌的车, 则该车会直接排到该品牌所有车辆的末尾; 否则, 新车直接排到所有车辆的末尾。

现在需要计算: 当一辆车到达并完成排队后, 它的前方还有多少车辆在等待。

(1) 假设在码头开放前, 已经有车辆 `[[1,0], [2,1], [3,2], [4,1], [5,0], [6,2]]` 按序到达, 列表中的每个元素包含两个数据项, 依次为车辆编号和品牌。码头开放后, 车辆 `[[7,0], [8,0], [9,2], [10,2], [11,1]]` 依次到达。例如, 当车辆 `[7,0]` 到达时, 7 号车可以直接排到 0 号品牌队伍的末尾, 此时 7 号车辆的前方还有 2 辆车 (正在装载的 1 号和 5 号车)。则当车辆 `[11,1]` 到达时, 11 号车辆的前方还有 __ 辆车。

(2) 定义如下 `arrive(car, brand, p)` 函数, 其功能是处理一辆新车到达码头时的排队并计算该车前方的车辆数量。参数 `car` 表示车辆编号, `brand` 表示品牌, `p` 表示当前节点索引。

```
def arrive(car, brand, p):
    global h, t
    front = 0
    for b in order[h:t]:
        if b == brand:
            break
        front += length[b]
    front += length[brand]
    if head[brand] == -1:
```

```

        head[brand] = p
        tail[brand] = p
        order[t] = brand
        t += 1
    else:
        data[tail[brand]][2] = p
        -----
        length[brand] += 1
    return front

```

请回答①和②两个问题：

① 请在划线处填入合适的代码。② 加框处代码 `head[brand] == -1` __ (选填：能/不能) 改为 `length[brand] == 0`。

(3) 提前到达的车辆和码头开放后到达的车辆信息存储在列表 `data` 中，`data` 的数据结构为：[[车辆编号，品牌，同品牌下一车辆索引]，...]，其中 `data[i][2]` 初始值均为 -1。如下 Python 程序用于模拟车辆到达排队过程，并计算每辆车到达时其前方等待的车辆数。请在划线处填入合适的代码。

```

h = t = 0
head = [-1] * m; tail = [-1] * m; length = [0] * m
m = 3
n = 100
order = [0] * n
def leave():
    global h
    brand = order[h]
    p = head[brand]
    -----①-----
    length[brand] -= 1
    if head[brand] == -1:
        tail[brand] = -1
    -----②-----
    return

def show_queue():
    #显示当前队伍情况及输出前面排队的车辆数，代码略

#处理码头开放前到达的车辆
for i in range(len(data)):
    car = data[i][0]
    brand = data[i][1]
    front = arrive(car, brand, i)
start = len(data)

```

```

new_cars = [[4, 2], [5, 1], [6, 0], [7, 1], [8, 2], [9, 0]]
#处理新到达的车辆序列 new_cars 放入 data 中，代码略
for i in range(start, len(data)):
    vid = data[i][0]
    brand = data[i][1]
    front = arrive(vid, brand, i)
    print("前面有"+str(front)+"辆车在队伍中")
    left = leave()
    show_queue()

```

答案

正确答案： (1) 2; (2) ① tail[brand] = p; ② 能; (3) ① head[brand] = data[p][2]; ② h += 1

解析

同品牌车辆插入到该品牌尾部，因此新尾指针要更新为 p；head[brand] == -1 与 length[brand] == 0 都表示该品牌当前无车。队首离开时，品牌头指针改为同品牌下一辆车；若该品牌队列空了，则品牌顺序队列头指针 h 后移。

C6：逻辑表达式栈运算

出处：[202501-宁波九校-高二-期末-15](#)。（来源：[202501-宁波九校-高二-期末.pdf](#)；难度：困难）

15. 小 z 同学自从学习了 Python 表达式后，一直在思考一个问题：计算机是如何计算表达式的？通过后缀表达式的学习，他想到了利用栈的思想也可以解决其他问题，由于是初学者，他选择了逻辑表达式进行尝试，并通过编写程序来模拟整个计算过程。

(1) 为了便于实现，输入的逻辑表达式一定是合法无误的，并且只包含 True、False、and、or、not 这五个逻辑值和运算符，请在划线处填入合适代码。

```

s = input("请输入逻辑表达式")
a = s.split()
b = [0] * (len(a)+1)
st = [False] * (len(a)+1)
top = -1
n = 0
①
for i in a:      # 小z发现not可以有多个并列出现，因此先去除了not
    if i == "not":
        flag = not flag
        ②
    elif flag:

```

```

        if i == "True":
            b[n] = "False"
        else:
            b[n] = "True"
        flag = False
    else:
        b[n] = i
    n += 1
a = b
for i in range(n):
    if a[i] == "True":
        top += 1
    ③
    if a[i] == "False":
        top += 1
        st[top] = False
    if ④:
        st[top-1] = st[top-1] and st[top]
        top -= 1
while top > 0:
    st[top-1] = st[top-1] or st[top]
    top -= 1
print(st[top])

```

(2) 假设输入一个不合法的表达式 `not True and not False and not and and`，则输出结果为__。

答案

正确答案： (1) ① `flag=False` 或 `flag=0`；② `n-=1`；③ `st[top]=True`；④ `a[i-1]=="and"` 或 `a[i-1]=="and" and i!=0`；(2) `False`

解析

变量 `flag` 用来记录当前是否有奇数个 `not` 需要作用到后面的逻辑值，初始应为 `False`。遇到 `not` 时只翻转 `flag`，不应把 `not` 存入处理后的列表，所以先执行 `n-=1` 抵消循环末尾的 `n+=1`。遇到 `True` 时入栈并设置 `st[top]=True`。`and` 的优先级高于 `or`，扫描到逻辑值后若前一个运算符是 `and`，就立即合并栈顶两个值。第 (2) 小题按该程序处理后，`not and` 中的 `and` 会被当成需要取反的普通非 `True` 标记处理为 `"True"`，最终栈内合并结果为 `False`。